

CM3070 Final Project

Project Title: Predicting Volume and Sentiment Surges in Reddit Financial Communities Using Machine Learning

Data Science Project Template: Predictive Modelling of Social Media Trend Emergence

Repository: <https://github.com/yentanh115/uol-bsc-cm3070-final-project.git>

Contents

1	Introduction (890/1000 words)	4
1.1	Background and Context	4
1.2	Problem Statement and User Needs	4
1.3	Aim, Objectives and Contributions	5
1.4	Prediction Target	5
1.5	Study Scope and Exclusions	6
1.6	Assumptions	6
1.7	Report Structure	7
2	Literature Review (1911/2500 words)	7
2.1	The Predictability of Online Attention	7
2.2	The Shift Toward Pre-Engagement Prediction	8
2.3	Sentiment as a Predictive Signal in Finance	9
2.4	Modelling Approaches for Attention Prediction	10
2.5	Financial Discussion on Reddit	11
2.6	Methodological Weaknesses in Prior Work	11
2.7	Research Gap and Project Position	12
3	Design (1914/2000 words)	13
3.1	Context, Requirements, and Acceptance Criteria	13
3.2	Data Selection Plan	15
3.3	Overall Pipeline Architecture	16
3.4	Data Representation and Preprocessing Design	18
3.5	Surge Definition (Target Variable)	18
3.6	Feature Engineering	20
3.7	Model Selection	21
3.8	Temporal Validation Design	22
3.9	Evaluation Framework	24
3.10	Experiment Plan	25
3.11	Design Trade-offs and Alternatives	26
3.12	Development Plan	27
4	Implementation (2500/2500 words)	28
4.1	Code Organisation	28
4.2	Exploratory Data Analysis Tooling	29
4.3	Data Loading and Preprocessing	32
4.4	Feature Engineering (Implementation)	33
4.5	Surge Labelling	35
4.6	Model Training	36
4.7	Evaluation Pipeline Implementation	40
4.8	Implementation Decisions Driven by Empirical Findings	40
4.9	Implementation Status	40
4.10	Testing Strategy	41
5	Evaluation (2170/2500 words)	43
5.1	Evaluation Against Project Objectives	43
5.2	Results	43
5.3	Critical Analysis	49
5.4	Limitations and Proposed Improvements	53
6	Conclusion (824/1000 words)	54

6.1	Current Achievements	54
6.2	Originality and Contribution	54
6.3	Key Findings	54
6.4	Limitations and Future Work	55
7	Appendices	56
7.1	Project Timeline	56
7.2	Software Dependencies	56
7.3	Repository Structure	57
8	References	58

1 Introduction (890/1000 words)

1.1 Background and Context

This project adapts the template of **CM3005 Data Science** (Predictive Modelling of Social Media Trend Emergence) to financial forums. On social media, a trend takes off when crowd attention suddenly converges on a specific topic or entity. This dynamic is especially intense in stock-market communities, where an obscure ticker can explode in popularity overnight, often signalling unusual trading activity before it happens (Long et al. 2023). Yet most current tracking tools only flag a trend once it is already widespread, by which point the chance to analyse, act on, or moderate the movement has usually passed. Anticipating these shifts before they peak is far more valuable than confirming them afterwards, but also harder, because it means recognising subtle patterns before the main surge arrives.

Turning that objective into a concrete task requires deciding both where to look and exactly what to predict. It focuses on Reddit’s financial communities, which uniquely combine public, archived, timestamped discussions tied to named stock tickers, so activity can be tracked at the level of an individual asset rather than the market wide. The core problem then comes down to this: given what a community has said about a ticker up to a given moment, *can we predict an imminent spike in its discussion?* Framed as a supervised classification task, the project forecasts short-term surges in ticker discussion before they peak.

1.2 Problem Statement and User Needs

What makes this prediction worth attempting is also what makes it hard. These spikes stem from earnings surprises, speculative momentum, or organized retail activity. Because they unfold over mere hours across thousands of daily tickers, manual tracking is nearly impossible.

This tension motivates the central **research question**: *can a social-media surge be predicted before it happens from early discussion patterns alone?* The answer matters to three user groups. **Market surveillance and compliance teams** would prioritise which tickers warrant investigation for unusual or potentially coordinated activity (FINRA 2025). **Quantitative researchers** would select a small set of tickers for deeper financial and textual analysis. **Platform moderators** would position monitoring capacity before discussion spikes. Across all three the system is a *prioritisation tool* that reduces a large candidate universe to a reviewable shortlist, not an autonomous decision-maker (Fawcett 2006). Their error tolerances differ, however: surveillance and moderation need broad coverage (high recall), whereas research can accept a smaller, higher-precision shortlist. These differing needs are formalised as acceptance criteria in Section 3.1.3 and tested in Section 5.3.5.

Beneath these differences lies one structural problem common to all three: candidate tickers vastly outnumber the capacity to review them, and surges develop faster than manual monitoring can react. Table 1 sets out the specific pain point each group faces and why current practice falls short.

Table 1: End-user pain points and the shortcomings of current practice.

User group	Pain point	Current workaround and its limit
Market surveillance / compliance	Thousands of tickers are discussed daily; potentially coordinated or unusual activity must be triaged, but surges develop too fast for manual review (FINRA 2025)	Rule-based volume alerts (e.g. $+2\sigma$) miss non-linear, multi-signal precursors and cannot rank candidates by likelihood

User group	Pain point	Current workaround and its limit
Quantitative researchers	A small set of tickers must be selected for deeper analysis from a large, noisy universe	Manual scanning is slow, subjective, and tends to surface tickers only after they have already become prominent
Platform moderators	Monitoring and moderation capacity must be positioned <i>before</i> discussion spikes, not after	Reactive moderation begins once a spike is already underway, when intervention is hardest

The shared need is therefore an *early, ranked shortlist* that compresses the candidate universe into a reviewable queue, leaving causal judgement and action to the human reviewer. This motivates the discrete, rankable target in Section 3.5 and the throughput-derived criteria in Section 3.1.3.

Meeting this need is not straightforward. Simple volume rules (e.g. $+2\sigma$) cannot capture non-linear signal interactions or rank candidates, and prior work has mostly targeted adjacent questions such as eventual reach or price movement, not short-term spikes in per-ticker discussion (Section 2). This project therefore takes a learning-based approach, integrating temporal, textual, and sentiment signals into one predictive framework.

1.3 Aim, Objectives and Contributions

To answer this question, the **aim** is to build and rigorously evaluate a system predicting per-ticker Reddit surges over a 24-hour horizon from only backward-looking features available at observation time. Three objectives, each with an explicit success test, follow (Table 2).

Table 2: Project objectives and their success tests.

#	Objective	-	Success test
O1	Predict surges	Build a model from early-stage discussion features (temporal, activity, sentiment) that forecasts surges before they occur	Model clears the target AUC-ROC tier (Table 19) on held-out future data
O2	Compare approaches	Establish whether more complex models improve prediction over simpler baselines	Model differences are quantified with uncertainty and significance (Section 5.2.3)
O3	Validate temporally	Establish whether predictions hold on unseen future periods under leakage-free evaluation	No feature, label, or statistic draws on future data, and results are reported on a chronological holdout (Section 5.1)

The **contributions** are a leakage-free forecasting methodology, a composite volume-and-sentiment surge metric, and evidence that data density constrains performance more than model complexity (Section 6).

1.4 Prediction Target

The template uses “trend emergence,” but trends can be gradual and sustained, making them hard to label objectively. This project instead targets *surges*: significant short-term spikes in both **posting volume** and **sentiment intensity** for a ticker **within a 24-hour window**, captured by a composite metric combining normalised volume growth with sentiment-shift magnitude (Section 3.5). Because surges are discrete and quantifiable, prediction becomes binary classification. The target uses timestamped post volume, not post-hoc engagement metrics like upvotes (which introduce look-ahead bias), with z-scores computed from

training statistics alone to prevent leakage. The underlying hypothesis is that backward-looking signal suffices to discriminate surges from baseline activity, and that performance scales with data density rather than model complexity.

1.5 Study Scope and Exclusions

The study evaluates binary surge classification on two archival 2021 Reddit datasets, *r/pennystocks* and *r/wallstreetbets* (hereafter **WSB**), chosen as sparse and dense communities. Three classifiers, Logistic Regression (**LR**), Random Forest (**RF**), and XGBoost (**XGB**), are trained on backward-looking features and evaluated on a chronological holdout with expanding-window cross-validation, bootstrap intervals, McNemar’s tests, single-feature baselines, and cross-dataset transfer, under a deterministic, seeded pipeline.

Excluded from scope: real-time ingestion and production deployment; user behaviours, comment networks, and cross-platform channels; multi-class or regression targets; and trading signals, financial advice, or causal claims about market impact.

1.6 Assumptions

The project’s conclusions hold under the explicit assumptions in Table 3, ordered from strongest to weakest. Stating them clarifies the boundary conditions for interpreting the results and signals where the analysis is most exposed.

Table 3: Explicit project assumptions and where each is addressed.

#	Assumption	Basis and where addressed
1	Human-in-the-loop deployment: the system prioritises candidates for a reviewer, not autonomous action	Underpins the acceptance criteria (Section 3.1.3); automation would need far higher precision and calibration
2	Bounded analyst throughput of ~20–30 flagged tickers per day	Anchors the daily-alert-volume criterion; a different capacity shifts the operating point, not the model
3	Ticker mentions proxy genuine discussion of a stock	Stopword-filtered regex extraction (Section 3.6) trades precision for recall, accepting minor mention noise
4	A 24-hour horizon is operationally relevant	Design choice aligned to the daily cycle, not an optimised horizon; multi-scale alternatives are future work (Section 5.4.2)
5	Backward-looking signal is sufficient, and performance scales with data density over model complexity	The core hypothesis, tested directly in Section 5.1
6	The 2021 archive is informative beyond its window	Weakest assumption; treated as a limitation (Section 5.4.2) and in the conclusion
7	Public forum text is usable in aggregate	Records are public submissions, analysed at ticker level with no individual user identification
8	Static, complete, well-ordered input data	Features and frozen statistics assume gap-free, chronologically ordered input with a

#	Assumption	Basis and where addressed
		stable schema, as in the archival dataset; a production stream (missing or late posts, schema drift, vocabulary shift) would require re-ingestion safeguards and periodic recalibration (Section 5.3.6)

1.7 Report Structure

Section 2 reviews the literature on online attention prediction, financial sentiment, and Reddit-specific research. Section 3 details the design: data selection, data preprocessing, surge definition, feature engineering, model selection, and temporal validation. Section 4 describes implementation. Section 5 presents results, statistical validation, critical analysis, and limitations. Section 6 concludes with key findings and future directions.

2 Literature Review (1911/2500 words)

2.1 The Predictability of Online Attention

Trend prediction spans many methods, from sequence and graph models to trajectory forecasting; this review focuses on the strand nearest to the project, supervised tabular classification from hand-crafted features. The sources reviewed here (Section 8) were selected because they directly inform the decisions this project makes: *what to predict* (popularity and surge-onset literature), *what signals to use* (sentiment and content features), *how to model* (classifier families and their trade-offs), and *how to evaluate rigorously* (temporal validation methods). A further strand, Reddit-specific financial research, confirms that this platform contains distinct, predictable signals.

The foundational question underlying this project, *can future surges in social media activity be predicted?*, was first addressed through research on **online popularity prediction**. This literature demonstrated that online attention is not random: *content that attracts early engagement tends to attract more, following patterns that are statistically detectable*.

Szabo and Huberman (Szabo and Huberman 2010) produced the seminal result in this domain, demonstrating strong linear correlations between early and later popularity on YouTube and Digg. Their regression model showed that a content item’s view count at time t predicts its eventual popularity with high accuracy. This established the core principle: early behavioural signals carry predictive information about future attention. However, because the model assumes a stationary growth process, it requires content to have already accumulated measurable engagement before prediction becomes possible. It cannot forecast at or near the time of posting.

Lerman and Hogg (Lerman and Hogg 2010) extended this understanding by modelling the interaction between social network structure and content discovery, demonstrating that popularity depends on behavioural dynamics beyond simple cumulative counts. Their agent-based approach revealed that network position and user browsing patterns mediate how content gains visibility. Wang and Huberman (Wang and Huberman 2013) and Kong et al. (Kong et al. 2014) further characterised online attention as following identifiable temporal lifecycles (emergence, growth, peak, and decline), suggesting that content at different lifecycle stages exhibits different observable signatures. Kong et al. (Kong et al. 2014) specifically addressed *burst*

detection for hashtags in real-time, but their approach operates at the hashtag level with contemporaneous features. It detects bursts as they happen rather than predicting them before onset, and works at a platform-wide granularity rather than per-entity (ticker) level.

The academic consensus that emerged from this first wave of research can be summarised as: *online attention is predictable from early signals, follows lifecycle dynamics, and is mediated by platform-specific network effects*. However, these models all require content to have already gained some traction before prediction is possible, and they target *eventual* popularity rather than the *onset* of rapid growth. Furthermore, a key tension exists within these findings: Szabo and Huberman (Szabo and Huberman 2010) show that early popularity strongly predicts final outcome, yet Cheng et al. (Cheng et al. 2014) later found that cascade prediction accuracy plateaus after the initial phase. This suggests that predictability diminishes once content leaves the emergence stage, which is precisely the window this project targets.

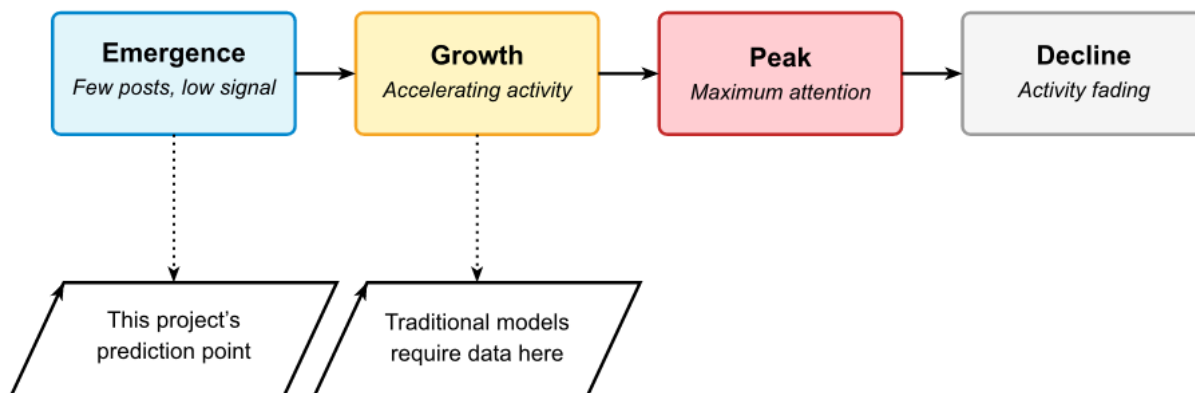


Figure 1: Online attention lifecycle model (Wang and Huberman 2013; Kong et al. 2014). Traditional popularity prediction requires content to have reached the growth phase before forecasting is possible. This project targets the emergence phase, predicting a surge before substantial engagement has accumulated.

2.2 The Shift Toward Pre-Engagement Prediction

A second wave of research addressed the limitation that early popularity models require existing engagement data. Bandari et al. (Bandari et al. 2012) demonstrated that content metadata such as source, category, subjectivity, and named entities could predict popularity before engagement accumulates, achieving approximately 84% classification accuracy on news articles. This represented a key methodological shift: prediction could occur at or before publication rather than requiring an observation period.

Cheng et al. (Cheng et al. 2014) achieved approximately 79.5% accuracy ($AUC = 0.877$) predicting whether Facebook photo cascades would double in size, using temporal features derived from early propagation speed and structural virality metrics. Their findings showed that the *rate* of initial spread, rather than its magnitude, carries predictive signal for sustained growth. Yuan and Li (Yuan and Li 2019) extended this principle across information diffusion contexts, showing that early-stage propagation patterns contain sufficient signal to forecast long-term diffusion trajectory.

This body of work established a second consensus: prediction is achievable before substantial engagement accumulates, provided features capture content characteristics or early propagation dynamics. However, the prediction targets remained eventual outcomes, such as final popularity or total cascade size, rather than *rapid onset* within a bounded time window. For example, an analyst monitoring a financial forum needs to know whether discussion will surge within the subsequent 24 hours, not whether it will eventually become popular. This temporal distinction represents a key unaddressed gap.

Taken together, the two waves point to a small set of signal families with predictive value, but the evidence for each is uneven and not all of them are usable under this project’s leakage constraint. Table 4 synthesises which signals prior work supports, how strong that evidence is, and which are admissible here (available strictly before the surge, computed from backward-looking data only).

Table 4: Predictive signal families, the strength of prior evidence, and their admissibility under this project’s backward-looking constraint.

Signal family	Evidence from prior work	Strength	Fit for this project
Volume / activity rate	Early view and post counts predict later attention (Szabo and Huberman 2010; Long et al. 2023)	Strong	Included as backward-looking counts (Section 3.6)
Growth / acceleration	Early <i>rate</i> of spread predicts sustained growth better than magnitude (Cheng et al. 2014; Yuan and Li 2019)	Strong	Included as lagged growth and acceleration features
Content meta-data	Source, category, named entities predict popularity pre-engagement (Bandari et al. 2012)	Moderate	Partially included (word/title counts, ticker counts)
Sentiment	Aggregate social sentiment informs financial forecasting (Bollen et al. 2011)	Moderate	Included as sentiment level and change (Section 5.2.6)
Network / diffusion structure	Network position and reshare structure mediate visibility (Lerman and Hogg 2010; Cheng et al. 2014)	Moderate	Excluded: requires cross-user graph data outside the per-ticker, per-post scope
Post-hoc engagement (up-votes, comments)	Correlates with popularity but accumulates <i>after</i> posting (Szabo and Huberman 2010)	Strong but unusable	Excluded: future information, would leak the target (Section 3.5)

2.3 Sentiment as a Predictive Signal in Finance

Alongside popularity research, computational finance studies established that collective social media sentiment carries measurable predictive information. Bollen et al. (Bollen et al. 2011) demonstrated that aggregate Twitter mood, particularly the “Calm” dimension measured by the Google-Profile of Mood States (GPOMS), predicted Dow Jones movements with about 87% directional accuracy. Although limited by a short evaluation window, missing out-of-sample testing, and an unclear causal mechanism, their study proved pivotal in establishing that **social media textual sentiment can inform financial forecasting**.

The tools used for sentiment extraction have evolved alongside this finding. General-purpose lexicons like OpinionFinder lack domain specificity for financial language, where terms like “short,” “bearish,” or “moon” carry specialised meaning. To better capture online discourse, Hutto and Gilbert (Hutto and Gilbert 2014) developed VADER specifically for social media text, incorporating rules for punctuation emphasis, capitalisation, degree modifiers, and negation, and achieving F1=0.96 on social media benchmarks. Araci (Araci 2019) later introduced FinBERT, a transformer model fine-tuned on financial corpora, capturing contextual meaning that rule-based tools miss. This progression from general lexicons, to social-media rules, to domain-specific deep learning highlights the field’s consensus that sentiment analysis tools must be tailored to their specific domain.

For this project, the key takeaway is that sentiment *change*, rather than absolute sentiment value, may serve as a leading indicator of activity surges: if a ticker’s discussion becomes markedly more emotional before volume escalates, sentiment shift could provide an early warning signal. This motivates including sentiment change magnitude in the composite surge metric.

Table 5: Evolution of sentiment analysis tools relevant to financial social media.

Tool	Type	Domain	Strengths	Limitations for This Project
OpinionFinder as used in (Bollen et al. 2011)	Lexicon	General	Early adoption, widely cited	No social media conventions, no financial terms
VADER (Hutto and Gilbert 2014)	Rule-based	Social media	Handles capitalisation, emoticons, negation; F1=0.96	No financial domain tuning (“short,” “moon” mis-scored)
FinBERT (Araci 2019)	Transformer	Financial text	Context-aware, domain-specific	Computationally expensive for 1M+ records

2.4 Modelling Approaches for Attention Prediction

How the prediction problem is *framed* varies across the literature, and the framing dictates the model family. Szabo and Huberman (Szabo and Huberman 2010) treat it as regression onto a continuous future count; Bandari et al. (Bandari et al. 2012) and Cheng et al. (Cheng et al. 2014) frame it as classification into popularity or cascade-growth classes; Kong et al. (Kong et al. 2014) and Yuan and Li (Yuan and Li 2019) adopt time-series or diffusion-trajectory formulations. Because this project predicts a discrete event, namely whether a surge occurs within a fixed window, binary classification is the natural framing, aligning it with the Bandari and Cheng strand rather than with continuous forecasting.

Within classification, prior work spans a complexity spectrum. Linear and regression models (Szabo and Huberman 2010; Bandari et al. 2012) are interpretable and cheap to fit, but assume roughly additive effects and can underfit interacting signals. Tree ensembles occupy the middle ground: bagging (Random Forest) averages independent trees to reduce variance, which is robust when positives are scarce, while boosting (gradient-boosted trees) fits residual errors sequentially and typically leads on structured tabular data when enough labelled examples exist. Fernández-Delgado et al. (Fernández-Delgado et al. 2014) found, across 121 datasets, that ensemble tree methods are consistently among the strongest general-purpose tabular classifiers, while also cautioning that no single family dominates and that evaluation methodology materially affects the ranking. Deep sequence models (e.g. the recurrent approaches implicit in trajectory work (Kong et al. 2014; Yuan and Li 2019)) can capture richer temporal structure but demand large labelled datasets and sacrifice interpretability, which is ill-suited to a sparse, highly imbalanced surge target where reviewers must be able to inspect why a ticker was flagged.

Two implications follow for this project. First, an interpretable linear model (Logistic Regression) is the appropriate baseline: if it performs well, the surge signal is largely additive; if a more complex model beats it, the margin quantifies what non-linear structure is worth. Second, tree ensembles (Random Forest and XGBoost) are the natural improved models, spanning the bagging/boosting contrast and letting the project test *empirically* whether added complexity pays off under scarce positives, rather than assuming the most

complex model is best. These choices are set out in Section 3.7 and the complexity-versus-data-density question is examined directly in Section 5.3.1.

2.5 Financial Discussion on Reddit

While the preceding research established foundational principles on platforms like YouTube, Digg, Facebook, and Twitter, recent studies examine whether these dynamics hold within Reddit’s financial communities and how their unique structural features alter information flow.

Penny stocks (low-capitalisation equities trading below \$5) occupy a distinctive position because their low liquidity and limited analyst coverage mean that social media discussion can constitute a disproportionate share of available information (Long et al. 2023; Costola et al. 2022). Unlike Twitter’s ephemeral broadcast environment, Reddit’s subreddit structure creates concentrated communities with persistent threads and shared behavioural norms.

Long et al. (Long et al. 2023) demonstrated that WSB posting volume correlated with abnormal trading volume and returns for discussed stocks, with effects concentrated in small-cap equities. Their analysis showed that increased Reddit attention *preceded* trading activity in their sample, suggesting that discussion patterns carry predictive signal rather than merely reflecting market events. Costola et al. (Costola et al. 2022) examined the GameStop episode specifically, finding that consensus formation within WSB followed measurable patterns in posting frequency and sentiment alignment *before* reaching critical mass. A small number of committed users drove broader engagement through detectable temporal signatures. Extending this to security manipulation, Mancini et al. (Mancini et al. 2024) constructed predictive models using the textual and temporal properties of forum posts. Their work confirmed that forum-derived text features significantly outperform chance baselines in forecasting anomalous stock activity.

These studies confirm that Reddit financial communities produce predictive signals, yet prior work exclusively targets market outcomes such as returns, volume, or manipulation, rather than platform dynamics. Notably, these findings align with general literature: Costola et al.’s consensus patterns (Costola et al. 2022) reflect Lerman and Hogg’s network discovery dynamics (Lerman and Hogg 2010), while Long et al.’s temporal precedence (Long et al. 2023) echoes Cheng et al.’s early speed metrics (Cheng et al. 2014). However, predicting whether the discussion itself will rapidly escalate, that is, forecasting an imminent surge in posting volume, remains an open challenge. Addressing this gap is the central focus of this paper.

2.6 Methodological Weaknesses in Prior Work

Beyond the substantive gaps identified above, a critical methodological pattern cuts across the literature: a recurring absence of rigorous temporal evaluation protocols.

This pattern recurs across the reviewed studies, as Table 6 catalogues: same-period evaluation, random train-test splits, random cascade sampling, and full-dataset correlation all break chronological ordering, allowing each model to be tested on data that would not have existed at prediction time.

This methodological oversight is significant because Tashman (Tashman 2000) demonstrated that rolling-origin evaluation, where the forecasting origin advances forward through time, produces far more reliable accuracy estimates for temporal prediction tasks than fixed or random splits. Furthermore, Bergmeir and Benítez (Bergmeir and Benítez 2012) showed empirically that random cross-validation overestimates predictive accuracy on time-dependent data, and recommended blocked or expanding-window schemes that preserve temporal ordering. Despite established best practices in time-series forecasting, they remain largely unadopted in social media prediction research.

Consequently, reported performance figures across the reviewed studies may be inflated by temporal leakage, and it remains uncertain whether models would generalise to genuinely unseen future periods. For any system intended for real-world deployment, including surge detection, this is a critical deficiency. As

Fernández-Delgado et al. (Fernández-Delgado et al. 2014) noted in their large-scale classifier benchmark, evaluation methodology substantially affects reported performance rankings, reinforcing that how a model is evaluated matters as much as which model is selected.

Table 6: Temporal evaluation practices across reviewed studies, with the specific flaw each introduces.

Study	Evaluation Method	Temporal Ordering Preserved?	Specific flaw	Leakage Risk
Szabo & Huberman (Szabo and Huberman 2010)	Same-period evaluation	No	Train and test drawn from the same period	High
Bandari et al. (Bandari et al. 2012)	Random train-test split	No	Tests on articles published before some training data	High
Bollen et al. (Bollen et al. 2011)	Fixed holdout (1 month)	Partial	Short window, no out-of-sample testing	Medium
Cheng et al. (Cheng et al. 2014)	Random cascade sampling	No	Cascades sampled without preserving time order	High
Long et al. (Long et al. 2023)	Full-dataset correlation	No	No test of generalisation to later periods	High
Costola et al. (Costola et al. 2022)	Full-dataset analysis	No	No test of generalisation to later periods	High
Mancini et al. (Mancini et al. 2024)	Chronological split	Partial	Partial ordering; limited holdout	Medium

2.7 Research Gap and Project Position

The literature reviewed above establishes four cumulative findings:

- Early behavioural signals predict future online attention (Szabo and Huberman 2010; Cheng et al. 2014)
- Prediction is possible before engagement accumulates, using content and propagation features (Bandari et al. 2012; Cheng et al. 2014; Yuan and Li 2019)
- Sentiment extracted from social media carries predictive value in financial contexts (Bollen et al. 2011; Hutto and Gilbert 2014)
- Reddit financial communities generate measurable signals that precede market activity (Long et al. 2023; Costola et al. 2022; Mancini et al. 2024)

Against these findings, four key gaps remain unaddressed, each of which this project targets directly (Table 7).

Table 7: The four research gaps and the design response to each.

#	Gap	What prior work lacks	How this project addresses it
1	Prediction target	Studies predict <i>eventual outcomes</i> (total popularity, cascade size, market returns), not the <i>onset</i> of rapid growth in a bounded window; no formulation for a composite volume-and-sentiment surge per entity	Composite surge metric defines a binary onset target within a strict 24-hour window (Section 3.5)
2	Signal integration	Each strand shows one feature category in isolation, temporal (Szabo and Huberman 2010), content (Bandari et al. 2012), sentiment (Bollen et al. 2011), structural (Cheng et al. 2014), with little empirical integration, despite evidence they interact (Lerman and Hogg 2010; Kong et al. 2014)	Feature set combines temporal, activity-frequency, sentiment, and textual signals (Section 3.6)
3	Domain specificity	General research (Szabo and Huberman 2010; Bandari et al. 2012; Cheng et al. 2014) neglects financial dynamics (event-driven reactions, domain language, speculation); Reddit finance work (Long et al. 2023; Costola et al. 2022; Mancini et al. 2024) predicts market consequences, not whether surges <i>occur</i>	Pipeline applied to two Reddit financial communities at opposite ends of the data-density spectrum
4	Temporal validity	Random or unspecified splits (Szabo and Huberman 2010; Bandari et al. 2012; Cheng et al. 2014; Long et al. 2023) may inflate reported performance; rigorous temporal methods (Tashman 2000; Bergmeir and Benítez 2012) remain unadopted in this domain	Expanding-window temporal cross-validation ensures no future information leaks into training

Whether this integration yields meaningful predictive performance is the empirical question examined in Section 5.

3 Design (1914/2000 words)

3.1 Context, Requirements, and Acceptance Criteria

Before any design detail, this subsection fixes what the system must achieve. Everything flows from the operational context: it sets the requirements, which in turn set a measurable acceptance bar.

3.1.1 Operational Context

The system is a daily screening tool, not an autonomous decision-maker: each cycle it ranks active tickers by predicted surge probability and surfaces the top- k for human review, as classifiers do in fraud detection and medical screening (Fawcett 2006; Vickers and Elkin 2006). This serves the three user groups (Table 1) and the research question (Section 1.2). Together with the objectives and assumptions (Table 3), it sets two kinds of requirement: functional and data-science.

3.1.2 Requirements

Functional requirements fix what the system does. *Data-science* requirements govern how it learns and is judged, chief among them that no feature, label, or statistic may draw on future data (Section 2.6). Table 8 consolidates both into six goals, tracing each from need, through requirement, to design response; the targets they are judged against follow in Table 19 and Table 9.

Table 8: Design goals, each tracing a user or domain need through the requirement it imposes to the design response that satisfies it.

#	User / domain need	Requirement	Design response
G1	Moderators must act before spikes; researchers and surveillance need lead time, not hindsight (Table 1)	Early prediction. Forecast a surge in the next 24 hours using only signals available when a post is scored	Forward-looking composite target over a 24-hour horizon (Section 3.5)
G2	Prior work is undermined by temporal leakage (Section 2.6); predictions must hold on genuinely unseen future data	No future information. No feature, label, or statistic may draw on data later than the record being scored	Backward-only features (Section 3.6), train-frozen z-scores (Section 3.5), time-ordered validation (Section 3.8)
G3	Reviewers must triage a shortlist and trust why each ticker was flagged (Section 3.1.3)	Ranked, interpretable output. Emit a per-ticker surge probability for ranking, on a transparent, inspectable feature basis	Probability ranking with top- k review, an interpretable baseline model, and permutation importance (Section 3.7, Section 3.1.3)
G4	A shortlist is only useful if it fits a reviewer’s daily capacity and catches enough genuine surges to be worth the effort	Operational usefulness. Meet the ranking, recall, precision, and alert-volume targets tied to analyst throughput	Acceptance criteria derived from workflow constraints (Table 9)
G5	Third parties must be able to verify the results	Reproducibility. Reproduce byte-identically from a fixed, public data snapshot and fixed seeds	Static archival dataset (Section 3.2) and a deterministic, seeded pipeline (Section 4)
G6	Objective 2 needs to know whether complexity actually helps, and by how much it could vary	Robust evaluation. Compare models against baselines with quantified uncertainty and significance on a chronological holdout	Expanding-window CV, bootstrap intervals, McNemar’s tests, and single-feature baselines (Section 3.8, Section 3.9)

The goals are not equally binding. **G2 (no future information) is the most important:** it is built into every stage, forbidding post-hoc engagement features, forcing training-only normalisation, and ruling out random cross-validation. The remaining goals fix what is predicted and how it is used (G1, G3), the bar for useful enough (G4), and how the result is produced and judged (G5, G6).

3.1.3 Acceptance Criteria

G4 sets operational usefulness as a goal; the concrete bar follows from the reviewer’s workflow. A surveillance analyst can review only a bounded number of flagged tickers per day, taken here as roughly 20–30 (Assumption 2, Table 3). On WSB, where hundreds of distinct tickers are discussed daily (577,872 ticker mentions across 2021, Table 29), the system must therefore compress that universe by at least an order of magnitude. Table 9 turns this constraint into measurable targets.

Table 9: Operational acceptance criteria, derived from the reviewer’s workflow (Assumption 2, Table 3) and the user scenarios (Section 1.2).

Criterion	Requirement	Rationale
Ranking quality (AUC-ROC)	≥ 0.80	True surges must appear near the top of the ranked list (Fawcett 2006)
Recall at operating threshold	≥ 0.50	Catch at least half of genuine surges (He and Garcia 2009)
Precision at operating threshold	≥ 0.10	At least 1 real surge per 10 flags (≤ 9 false alarms per true positive); precision-recall analysis is the right lens under severe imbalance (Saito and Rehmsmeier 2015)
Daily alert volume	20–30 flags	Matches analyst throughput

These criteria are shaped by two considerations. First, errors are asymmetric: a missed surge costs more than an unnecessary review. A compliance team that misses a pump-and-dump faces regulatory risk, whereas investigating a benign ticker costs only analyst-hours. This favours recall-oriented thresholds and cost-sensitive learning (Elkan 2001; He and Garcia 2009). Second, the prediction is deliberately narrow: a high surge probability means discussion is likely to escalate within 24 hours, not that price will move or manipulation is occurring, so the system flags candidates while humans judge cause and response (Vickers and Elkin 2006). The criteria therefore assume human-in-the-loop review; fully automated action would demand precision ≥ 0.80 and formal probability calibration. Whether the built system meets even the human-in-the-loop bar is tested in Section 5.3.5.

With the context, requirements, and acceptance bar established, the remaining subsections work through the design in pipeline order, each returning to the goal it serves. One prerequisite comes first: the design can only be built on data that actually exists and can support a surge label, so Section 3.2 establishes that data before the architecture is settled.

3.2 Data Selection Plan

The design assumes a specific dataset, so before any of it is fixed the project must establish what data is actually reachable and whether it can support a surge label at all. This is settled in a dedicated data-understanding phase (Phase 3, Table 23) of exploratory data analysis (EDA). It is a project activity that feeds the design rather than a stage of the running system: kept separate from the pipeline, it is a screening aid that answers three questions in turn and records why rejected options failed. Its tooling and results are reported in Section 4.2. Table 10 sets out each decision, the criteria applied, and what disqualifies a candidate.

Table 10: EDA screening decisions, the criteria applied, and what disqualifies a candidate.

Decision	Criteria applied	Disqualifies a candidate
Which platform	Public (no auth), sub-hourly timestamps, per-ticker attribution, sufficient per-ticker volume, static archival snapshot	No engagement fields or ticker attribution, so no surge label is possible, whatever the size (rules out Twitter/X vs Reddit)
Which communities	Two communities at opposite posting densities (for abundance-vs-scarcity and cross-dataset transfer); screened on ticker diversity, volume surviving extraction, selftext availability	Long-form posting that defeats per-ticker extraction, or single-ticker focus that makes the design trivial
Whether a surge signal exists	Viability gate: surge-label fields present and at least one candidate definition yields a learnable positive class; data quality, coverage, and sentiment reliability recorded	No surge-label fields, or no definition yields a viable positive class

The viability gate uses a deliberately simple surge heuristic on sampled data, distinct from the leakage-free composite target used for actual labelling (Section 3.5), so its sample-level statistics differ from the pipeline’s full-run figures.

Ethics and known limitations. All data is publicly posted forum submissions, analysed only at ticker level with no individual user identified. The main limitations are inherited from the source: deleted posts are absent (survivorship bias), engagement metrics are frozen at their final values, and findings are tied to the 2021 period the archive covers.

3.3 Overall Pipeline Architecture

With the dataset established in Section 3.2, the prediction system is designed as a six-stage linear pipeline (Table 11) that ingests those selected Reddit submissions. Each stage consumes the previous stage’s output and writes intermediate artefacts to disk, enabling independent re-execution without recomputing upstream operations.

Table 11: Six pipeline stages and their responsibilities.

#	Stage	Responsibility
1	Data Loading and Preprocessing	Ingest raw CSV, clean text, extract tickers by regex, explode multi-ticker records into record–ticker pairs
2	Temporal Windowing	Compute per-ticker forward and backward 24-hour posting counts via vectorised binary search
3	Sentiment Computation	Compute VADER compound scores per record, with title-fall-back when selftext is absent
4	Target Labelling	Temporal 80/20 split, z-score parameters from training statistics only, binary targets via composite thresholding
5	Feature Engineering	Extract eleven backward-looking features (Section 3.6)
6	Model Training and Evaluation	Expanding-window cross-validation, hyperparameter tuning, holdout evaluation, statistical testing

Data Pipeline Architecture

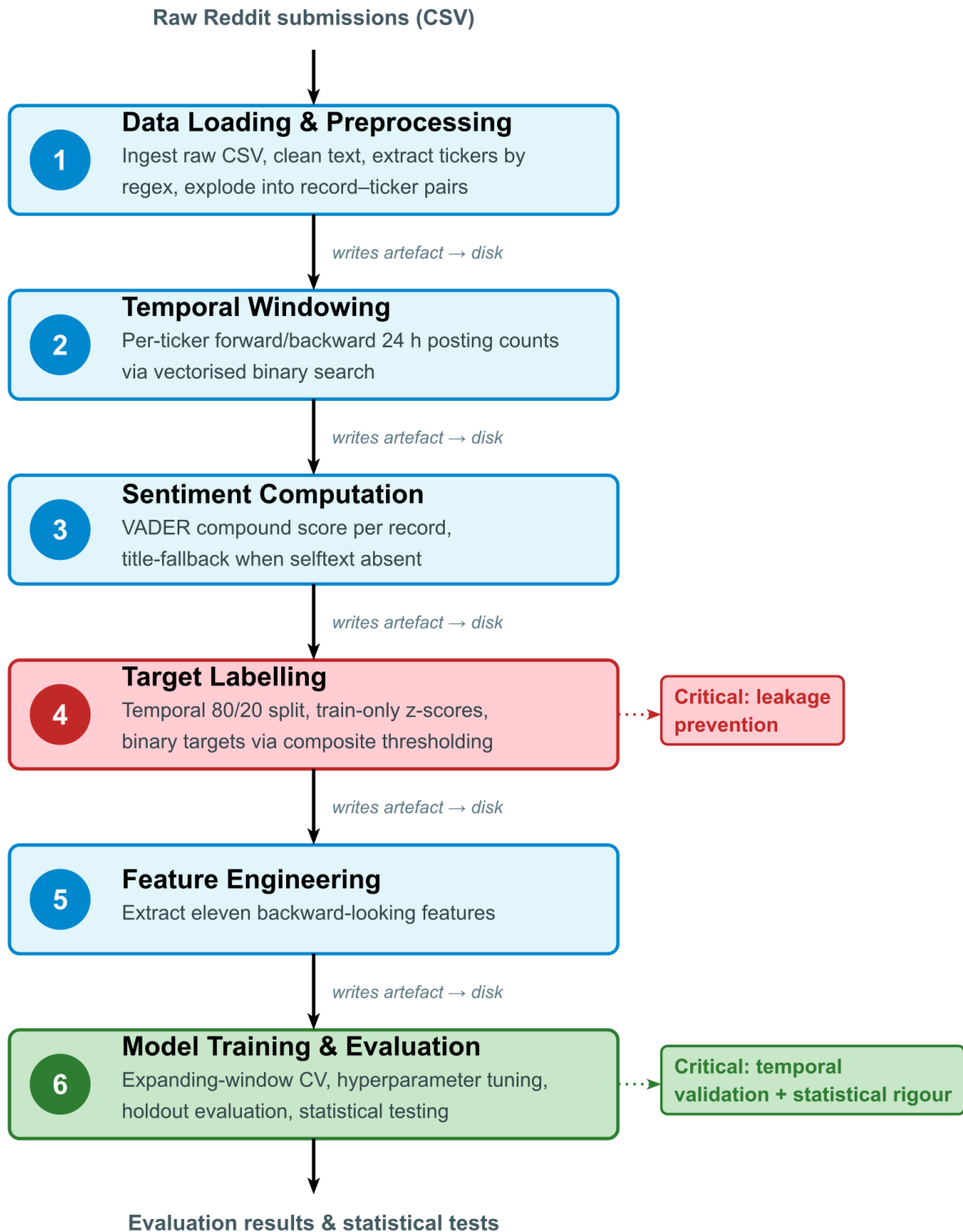


Figure 2: Pipeline architecture. Shading indicates critical design points: target labelling (leakage prevention), model training (temporal validation), and evaluation (statistical rigour).

Every stage honours goal G2: no stage may access future information relative to a record’s observation time. How this is enforced at each stage, backward-only windows, train-frozen z-scores, and time-ordered folds, is detailed in the subsections that follow.

3.4 Data Representation and Preprocessing Design

With the source dataset selected (Section 3.2), the first design step is to fix how a raw submission from it becomes a modelling unit, before any target is defined on it. A single post can mention several tickers, but a surge is defined *per ticker*, so the **unit of analysis is the record–ticker pair**: a post naming three tickers becomes three rows, each carrying the post’s text, timestamp, and the one ticker it is attributed to. This makes per-ticker windowing and labelling well defined and keeps a multi-ticker post from being forced into a single entity. Four further representation choices follow from this unit, each resolving how a specific aspect of a raw submission is encoded (Table 12).

Table 12: Representation choices for encoding a raw submission, each with its rationale and trade-off.

Aspect	Design choice	Rationale and trade-off
Temporal granularity	Keep native sub-hourly timestamps; measure windows relative to each record’s own observation time ($[t - 24 \text{ h}, t)$ backward, $(t, t + 24 \text{ h}]$ forward)	Event-centred rather than grid-centred, so no arbitrary calendar bin splits a surge and one 24-hour definition applies to dense and sparse tickers alike
Entity identification	Extract tickers from post text (the archive carries no ticker field); treat a mention as a <i>proxy</i> for genuine discussion (Assumption 3, Table 3)	Text extraction is noisy, but trading precision for recall keeps emerging or rotating tickers from being silently dropped
Missing periods	Do not impute gaps; an absent interval contributes a zero to a backward or forward count. Records whose forward window holds fewer than two same-ticker posts are excluded from labelling (Section 3.5)	A zero is the honest representation of “no discussion” and avoids inventing activity; too-sparse forward windows cannot define a meaningful growth ratio, so they are excluded rather than guessed at
Text, sentiment, engagement	Normalise text (moderation placeholders and nulls emptied) before extraction; represent sentiment as one per-record VADER compound score with title-fallback; exclude engagement fields (upvotes, comments) from features	Title-fallback gives every record a defined value without a separate imputation step; engagement is post-hoc and would leak future information, so it informs only dataset selection, never the model

Taken together, per record–ticker pair, event-relative windows, one sentiment scalar, and no engagement, this is the minimal representation that supports a leakage-free per-ticker surge label while remaining computable at scoring time.

3.5 Surge Definition (Target Variable)

A single fixed posting-count cutoff cannot compare surges across tickers, because the same absolute count means different things for different baselines: twenty posts in an hour is explosive for an obscure penny stock but routine for a heavily discussed ticker. The target therefore measures *relative* volume growth, normalised against the training distribution, and combines it with sentiment change into a composite score. For each record mentioning ticker X at time t , the pipeline computes the composite in five steps (Table 13).

Table 13: The five-step composite surge computation.

Step	Operation	Definition
1	Window counts	Count posts mentioning ticker X in the backward window $[t - 24 \text{ h}, t)$ and forward window $(t, t + 24 \text{ h}]$, giving C_{bwd} and C_{fwd}
2	Volume growth	$\Delta V = C_{\text{fwd}} / \max(C_{\text{bwd}}, 1) - 1$
3	Sentiment shift	$\Delta S = \bar{S}_{\text{fwd}} - s_t $, where $\bar{S}_{\text{fwd}}$ is the mean VADER score over forward-window posts and s_t the current post's score
4	Standardise	$Z(\Delta V), Z(\Delta S)$ using training-partition μ, σ exclusively
5	Composite + label	Composite = $w_1 Z(\Delta V) + w_2 Z(\Delta S)$; label surge ($y = 1$) if Composite $> \tau$

Computing μ_{train} and σ_{train} strictly from the training partition (step 4) prevents test-set distribution information from leaking into the target labels.

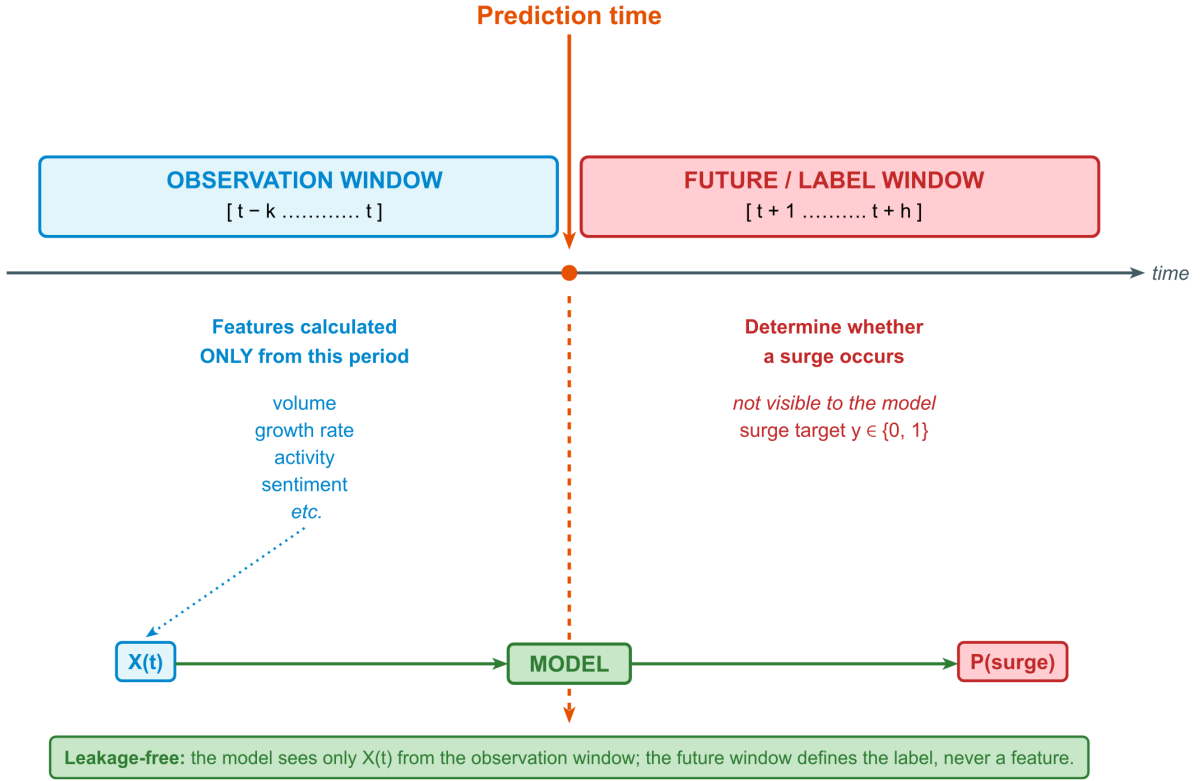


Figure 3: Observation, prediction, and surge-label windows. Every feature is computed only from the backward observation window $[t - k, t]$; the forward window $(t, t + h]$ is used solely to determine the surge label $y \in \{0, 1\}$ and is never visible to the model. This separation is what makes the target leakage-free.

The definition has three free parameters, set once and held fixed across the pipeline (Table 14).

Table 14: The three parameters of the surge definition and their fixed values.

Parameter	Value	Rationale
Observation window	24 h	Aligns with daily trading cycles. Shorter (6h) is expected to give sparse, unstable counts and longer (72h) to blur surge onset; these are design rationales, not tested outcomes, and multi-scale windows are future work (Section 5.4.2)
Weights w_1, w_2	0.5, 0.5 (composite)	ΔS is included to catch discussion that grows polarised before volume spikes (Elkan 2001; Bandari et al. 2012); whether it earns its place is tested against a volume-only target ($w_2 = 0$) in Section 5.2.6, and the VADER signal behind ΔS is screened in the EDA (Section 4.2)
Threshold τ	1.5 (primary), 1.0 (secondary)	τ trades anomaly purity against class balance: higher isolates rarer, clearer surges but leaves fewer positives. $\tau = 1.5$ balances the two; the EDA viability gate (Section 3.2) confirms a workable positive class survives, and surge counts across τ appear in Section 5.1

3.6 Feature Engineering

All eleven features satisfy a strict backward-looking constraint: each is derived exclusively from information available at or before observation timestamp t . Post-hoc engagement metrics (Reddit score, num_comments) are excluded because they accumulate after publication and would introduce lookahead bias.

Table 15: Feature definitions. All features use backward-looking or concurrent information only.

#	Feature	Category	Definition
1	sentiment_score	Content	VADER compound sentiment of the post text
2	word_count	Content	Words in selftext (0 if absent)
3	title_length	Content	Character count of title
4	num_tickers_mentioned	Content	Distinct tickers extracted from the post
5	hour_of_day	Temporal	Hour of post creation (UTC)
6	day_of_week	Temporal	Day of post creation (Monday=0)
7	time_since_previous	Activity	Seconds since previous same-ticker post
8	ticker_post_rate_24h	Activity	Same-ticker posts in preceding 24 hours
9	ticker_post_acceleration	Activity	Rate ratio: count in preceding 12h $\div$ count in 12h before that
10	word_count_x_hour	Interaction	$\text{word_count} \times \text{hour_of_day}$
11	accel_x_time_since_prev	Interaction	$\text{ticker_post_acceleration} \times \text{time_since_previous}$

The four categories (content, temporal, activity, interaction) draw on the signal families supported by the literature (Table 4); activity features in particular operationalise the popularity-prediction evidence that early volume and growth rate predict later attention (Szabo and Huberman 2010; Cheng et al. 2014). The two interaction terms are cross-feature products that give the models an explicit signal for combined dynamics, such as long analytical posts arriving at peak trading hours, without relying on deep tree splits to discover them; their empirical value is assessed by ablation (Section 4.4).

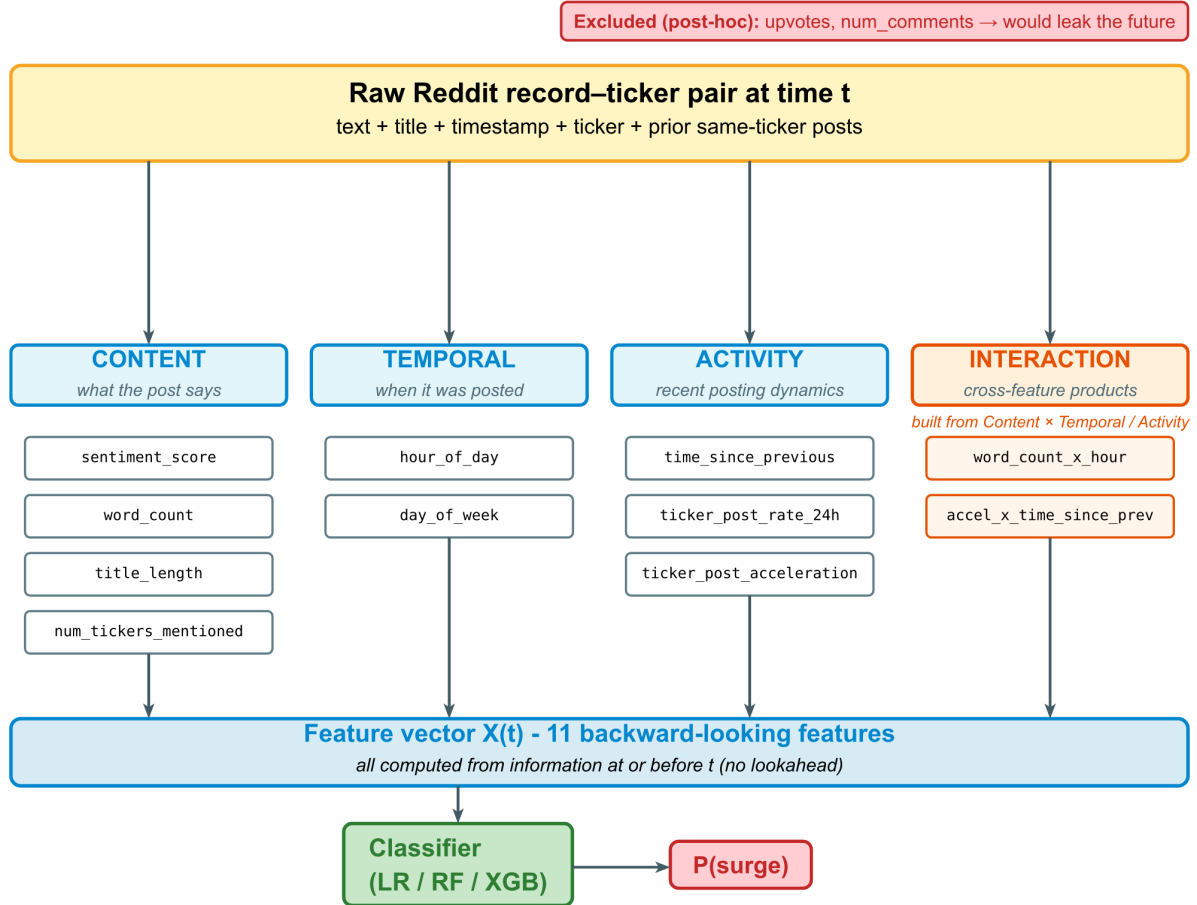


Figure 4: Feature engineering architecture. A raw record–ticker pair observed at time t is transformed into eleven backward-looking features across four categories (content, temporal, activity, interaction), assembled into the feature vector $X(t)$ and passed to the classifier. Post-hoc engagement signals (upvotes, comments) are excluded because they would leak future information.

3.7 Model Selection

The prediction objective is a supervised binary classification task ($y \in \{0, 1\}$): whether a ticker experiences a composite surge within the subsequent 24 hours. Three classifier families spanning the complexity spectrum (Table 16) evaluate whether architectural sophistication improves prediction.

Table 16: Three classifier families spanning the complexity spectrum.

Model	Role	Rationale
Logistic Regression (LR)	Interpretable linear baseline (Elastic Net, $L_1 + L_2$)	Strong performance would indicate approximate linear separability of the surge feature space
Random Forest (RF)	Bagged tree ensemble	Captures non-linear relationships; consistent top-tier tabular performance (Fernández-Delgado et al. 2014)
XGBoost	Gradient-boosted trees (L_1/L_2 leaf-weight regularisation)	Each tree corrects prior errors; boosting is among the strongest tabular families when labelled positives are sufficient (Fernández-Delgado et al. 2014)

Primary metric (AUC-ROC): At the sub-1% to ~5% surge rates the target actually produces (Table 29, Table 43), accuracy is uninformative, a naive “no surge” predictor scores 95–99%. AUC-ROC measures ranking quality across all thresholds. Precision, Recall, and F_1 are reported as secondary metrics at default and validation-optimised thresholds.

Handling class imbalance: SMOTE is unsuitable for temporal data because synthetic instances lack meaningful timestamps and risk local data leakage. Instead, imbalance is addressed via cost-sensitive learning: `class_weight='balanced'` for LR and RF, and `scale_pos_weight` (negative-to-positive ratio) for XGBoost.

3.8 Temporal Validation Design

Standard k -fold cross-validation violates chronological ordering by permitting models to train on future observations while validating on past ones, systematically overestimating performance (Bergmeir and Benítez 2012). The evaluation pipeline employs the two-level temporal partitioning scheme in Table 17.

Table 17: Two-level temporal partitioning scheme.

Level	Scheme	Partitioning	Temporal guarantee
1	Train/test split (80/20 by timestamp)	Records sorted by observation timestamp; earliest 80% form the training partition, final 20% the held-out test set, evaluated exactly once	Every test record occurs strictly after every training record
2	Expanding-window CV within training ($k = 4$)	Training partition split into four chronological blocks, producing three validation splits; each fold trains on all preceding blocks and validates on the next	Every validation instance occurs strictly after all training instances in its fold

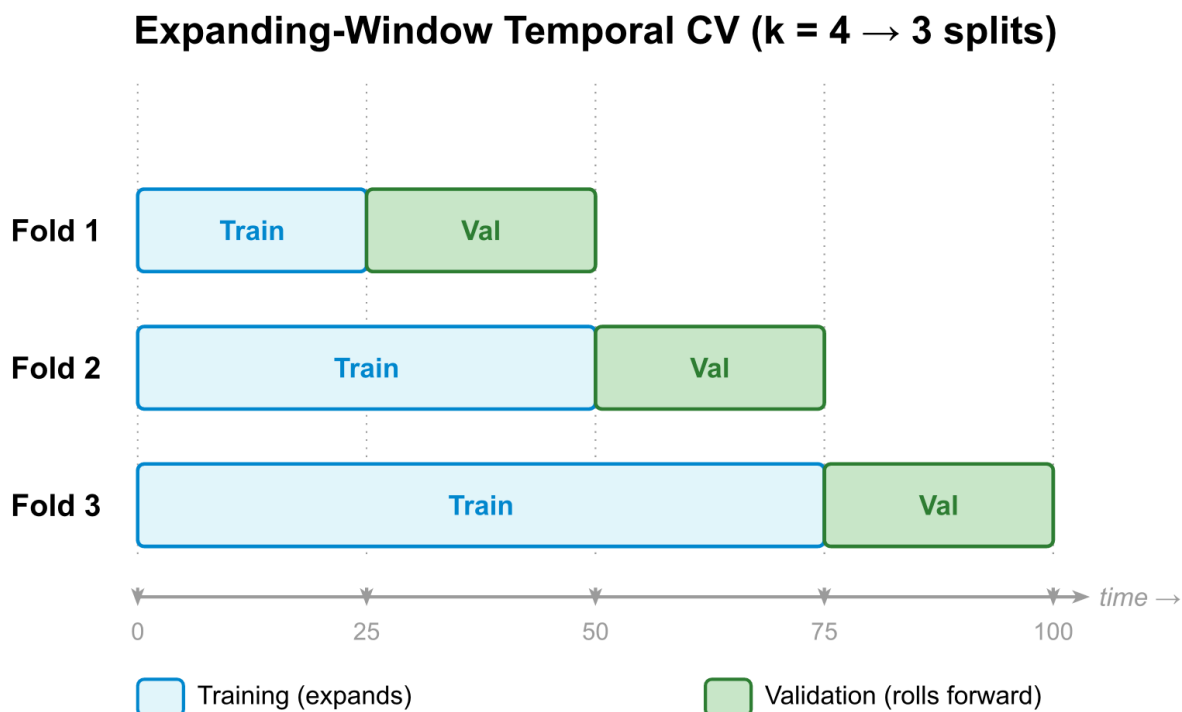


Figure 5: Expanding-window CV. The training partition is divided into four temporal blocks, producing three validation splits. Each fold trains on all data up to a cutoff and validates on the next block, mimicking deployment where more history accumulates over time.

A fold count of $k = 4$ is chosen to balance two competing needs: enough positive surge instances per validation window for stable AUC estimation, and adequate initial training depth in the first fold. Following hyperparameter optimisation, F_1 -optimised decision thresholds are locked on validation folds and applied unchanged to the test set, ensuring uncontaminated final evaluation. Temporal non-stationarity (shifting community behaviour across 2021) is mitigated by the expanding-window design but remains a structural risk; empirical evidence is detailed in Section 5.3.6.

Figure 6 situates this two-level partitioning within the end-to-end development workflow: models are developed and tuned using only the training partition and its expanding-window folds, the single winning configuration is retrained on the full training partition, and it is then scored exactly once on the strictly-later held-out test set before analysis.

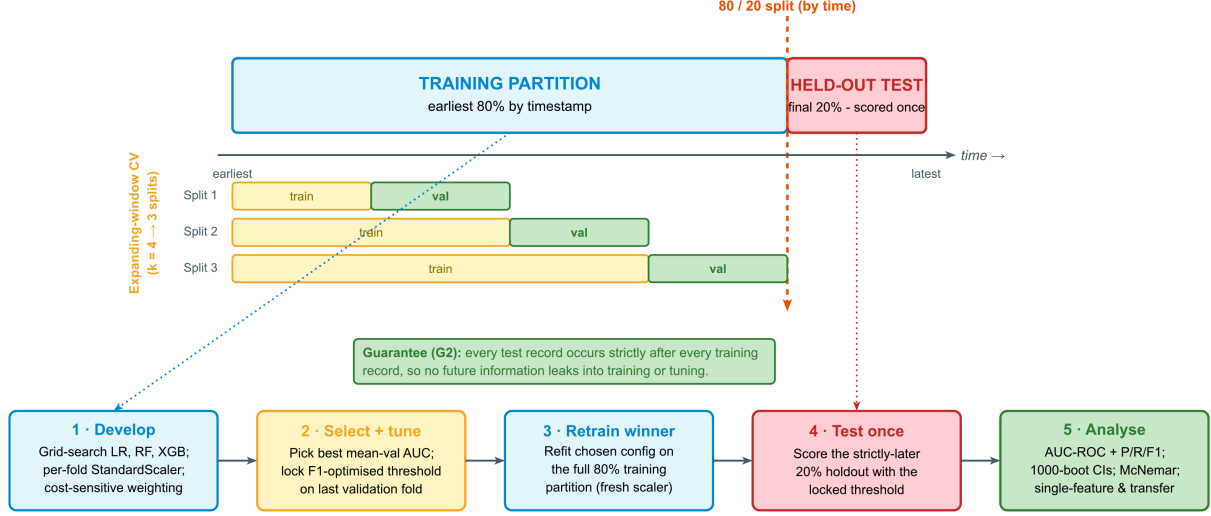


Figure 6: Chronological model development and evaluation workflow. The 80/20 split and expanding-window CV (levels 1 and 2 of Table 17) feed a develop $\rightarrow$ select and tune $\rightarrow$ retrain $\rightarrow$ test-once $\rightarrow$ analyse pipeline. Every test record occurs strictly after every training record (goal G2), so no future information leaks into training, tuning, or threshold selection.

3.9 Evaluation Framework

This subsection specifies the protocol by which the built system is judged (the detailed form of goal G6), applied to the held-out test set in Section 5. It fixes four evaluation questions and the method that answers each (Table 18).

Table 18: Evaluation questions and the method answering each.

#	Question	Method	Reported
Q1	Do models beat trivial base-lines?	Compare against a random baseline (AUC 0.50) and eleven single-feature Logistic Regression models	Section 5.2.3
Q2	Do ensembles beat the linear baseline?	McNemar’s pairwise test, Bonferroni-corrected $\alpha = 0.017$ ($0.05/3$)	Section 5.2.3
Q3	How confident are the estimates?	1,000 bootstrap resamples of the test set for 95% confidence intervals	Table 35
Q4	Does it generalise across communities?	Train on one subreddit, test on the other without retraining; a chosen transfer threshold of $\text{AUC} > 0.60$ (comfortably above the 0.50 chance level) is read as evidence of shared structure	Section 5.3.4

Ranking quality is scored against the tiers in Table 19, using the metrics in Table 20. These tiers gauge *research-level* discrimination and are distinct from the stricter, workflow-derived acceptance criteria (Section 3.1.3), which gauge *operational* usefulness. Robustness is further stress-tested by two sweeps: threshold ($\tau \in \{0.5, 1.0, 1.5, 2.0, 2.5\}$) and sentiment weight ($w_2 \in \{0.0, 0.25, 0.50, 0.75, 1.00\}$).

Table 19: Success tiers.

Tier	AUC-ROC	Interpretation
Minimum	> 0.60	Weak but above-chance discrimination
Target	> 0.70	Moderate; practically useful for ranking
Stretch	> 0.80	Strong; reliably separates surges from non-surges

Table 20: Evaluation metrics.

Metric	Role
AUC-ROC	Primary; threshold-independent ranking quality
Precision	Proportion of predicted surges that are real
Recall	Proportion of actual surges detected
F1-Score	Harmonic mean of precision and recall

3.10 Experiment Plan

The framework above fixes *how* a run is judged; this subsection fixes *which* runs are performed. The study is a matrix of configurations, each varying one factor (dataset, sentiment weight, threshold, seed, or transfer direction) around a fixed baseline, so every result is attributable to a deliberate change. Each carries a short code that the Implementation and Evaluation sections cite when reporting results (Table 21).

Table 21: Planned experiment matrix, grouped by the question each set of runs answers.

Group	Config(s)	Held fixed / varied	Question it answers
Baselines	A1 (r/pennystocks), A2 (WSB)	Default settings ($\tau = 1.5$, $w_1 = w_2 = 0.5$, seed 42); dataset varied	Can surges be predicted on sparse vs dense data? (O1, Sec- tion 5.1)
Sentiment contribu- tion	B1/B3 (volume-only, $w_2 = 0$) vs baselines	Weight varied to $w_2 = 0$ on each dataset	Does sentiment in the target im- prove over volume alone? (Sec- tion 5.2.6)
Weight sensitivity($w_2 \in \{0, 0.25, 0.5, 0.75, 1.0\}$)	Baseline + G-series on WSB)	Sentiment weight swept; dataset and τ fixed	How does the volume/senti- ment balance affect predictabil- ity? (Table 43)
Threshold sensitivity	C1 (WSB), C2 (r/ pennystocks), $\tau = 1.0$	Threshold lowered from 1.5 to 1.0	How does the surge threshold affect class balance and perfor- mance? (Section 5.1)
Robust- ness	Baseline + 4 additional seeds (r/pennystocks)	Seed varied over {42, 123, 456, 789, 2024}	Are results stable across random seeds?
Cross- dataset transfer	D1 (WSB→r/ pennystocks), D2 (r/ pennystocks→WSB)	Train community and test community swapped, no retraining	Do surge patterns generalise across communities? (Q4, Sec- tion 5.3.4)

Where groups overlap (the volume-only and full weight sweep coincide at $w_2 = 0$), the shared point is reused rather than re-run. All runs use the same seeded, deterministic pipeline (Section 4), each logged with its configuration and Git commit for traceability.

3.11 Design Trade-offs and Alternatives

Several plausible design choices were considered and deliberately not taken. Table 22 records each alternative, why it was rejected, and the constraint that drove the decision, so the chosen design is legible as a set of trade-offs rather than defaults. The unifying theme is feasibility under a fixed archival dataset, a strict no-leakage requirement, and a single-developer time and compute budget: where an option added capability at the cost of leakage risk, scope creep, or data the archive cannot supply, it was set aside.

Table 22: Design alternatives considered and the constraints behind rejecting each.

Alternative considered	Why not chosen	Governing constraint
Real-time ingestion / live dashboard	Adds streaming infrastructure orthogonal to the research question; retrospective evaluation answers it more cleanly	Scope, time; deferred to future work (Section 5.4.2)
Network / diffusion features (user graphs, reshare cascades)	Archive has no reliable user-interaction graph; would break the per-ticker, per-post unit of analysis	Data availability, scope (Table 4)
LSTM / sequence or time-series models	Sparse, highly imbalanced positives; large labelled-data and compute demands; opacity conflicts with the interpretability requirement	Data density, compute, interpretability (Section 2.4)
SMOTE / synthetic oversampling	Synthetic points lack meaningful time-stamps and risk local temporal leakage	No-leakage requirement (Section 3.7)
Post-hoc engagement features (upvotes, comments)	Accumulate after posting; using them would leak the outcome	No-leakage requirement (Section 3.4)
Volume-only surge target	Misses cases where emotional charge rises before volume; tested but retained only as a phase-1 control	Answered empirically, not assumed (Section 5.2.6)
Longer / shorter windows (6h, 72h) or multi-scale	6h expected too sparse for stable statistics; 72h expected to blur onset; multi-scale adds tuning surface beyond budget	Statistical stability, time (Section 3.5, Section 5.4.2)
Closed exchange-listed ticker universe	Penny and emerging tickers rotate frequently; a fixed list silently drops unknown stocks	Recall over precision (Section 3.4)
Multiple platforms (Twitter/X, StockTwits)	No engagement/attribution fields to support a surge label; would fragment the study	Data availability (Section 3.2)

Taken together, the design trades breadth for defensibility: a narrower, fully leakage-free, reproducible pipeline on two contrasting communities is preferred over a broader system whose results could not be trusted or reproduced. The most consequential trade-off is the exclusion of network and post-hoc engagement signals; both are known-predictive in prior work but inadmissible here, which bounds the achievable performance and is revisited in the limitations (Section 5.4.2).

3.12 Development Plan

Table 23 outlines the main project phases, activities, and expected deliverables.

Table 23: Project timeline and deliverables.

Id	Phase	Key Activities	Deliverables
Phase 1	Business Understanding & Scoping	Define problem, users, research question, scope and success criteria	Project definition and requirements
Phase 2	Literature Review	Review trend prediction, engagement prediction and sentiment analysis research	Literature review and research gap
Phase 3	Data Understanding	Dataset investigation, exploratory analysis and quality assessment	Dataset profile and EDA results
Phase 4	System Design	Design prediction pipeline, feature set, surge definition and evaluation strategy	System architecture and design specification
Phase 5	Prototype Development	Implement baseline pipeline and Logistic Regression model	Feasibility prototype
Phase 6	Full Pipeline Implementation	Preprocessing, feature engineering and advanced models	Complete predictive system
Phase 7	Evaluation & Analysis	Model comparison, cross-validation, error analysis and feature importance analysis	Evaluation results
Phase 8	Refinement	Improve models and address identified weaknesses	Refined prototype
Phase 9	Final Report & Demonstration	Final documentation and demonstration video	Final report and video

The plan is derived from the CRISP-DM data-mining process model, whose stages (business understanding, data understanding, modelling, evaluation) map directly onto Phases 1–9; this grounding ensures the schedule follows an established methodology rather than an ad-hoc ordering. The phases are sequenced by dependency: scoping and the literature review (Phases 1–2) fix the research question and success criteria that the system design (Phase 4) must satisfy, and data understanding (Phase 3) constrains the surge definition and feature set before any modelling begins. A feasibility prototype (Phase 5) is scheduled ahead of full implementation specifically to de-risk the approach, validating the leakage-free labelling and a single baseline model before committing effort to the complete pipeline. The process is iterative rather than strictly linear: evaluation (Phase 7) feeds refinement (Phase 8), which loops back through implementation and re-evaluation as weaknesses such as threshold miscalibration and data sparsity are identified and addressed.

This design allocates the most time to the implementation and evaluation phases, where the project’s technical risk is concentrated. The corresponding schedule is shown as a Gantt chart in Figure 13.

4 Implementation (2500/2500 words)

4.1 Code Organisation

The pipeline is packaged as a standard Python 3.10+ library (surge-pipeline, built with setuptools via pyproject.toml). Dependencies are declared as minimum-version ranges in both pyproject.toml and requirements.txt (listed in full in Section 7.2), and the resolved environment is logged with each experiment run so a run can be reproduced against the versions actually used.

The pipeline source resides under src/ (Listing 1), split into core library modules and executable CLI scripts. (see Section 4.2). The surge_pipeline/ package holds one module per pipeline stage (16 modules, ~4,450 LOC), plus a few shared-utility (timestamps.py, cli_logging.py) and data-contract modules (training_models.py, evaluation_models.py), with a matching test file per stage under tests/. Alongside it, a separate top-level eda/ directory holds the standalone data-selection notebooks, kept outside the pipeline package and sharing no code with it. The full repository tree is given in Section 7.3.

Listing 1: Repository code organisation.

```
src/
├── surge_pipeline/           # Core library (16 modules, ~4,450 LOC)
│   └── ...
├── tests/                   # 10 test modules (pytest)
│   └── ...
│   ├── run_labeling.py      # CLI: full labelling pipeline (stages 1–4)
│   └── run_training.py      # CLI: model training + evaluation (stages 5–6)
...
eda/                         # Standalone EDA notebooks
└── ...
```

This modular separation ensures that changes to one stage (e.g., swapping out the sentiment backend) cannot touch another’s logic, and any stage can be unit-tested in isolation.

Executable commands are exposed via entry-point CLI scripts (declared in pyproject.toml) to streamline individual stages and end-to-end runs.

Table 24: CLI entry points.

Command	Purpose
surge-label	Run the labelling pipeline (load -> window -> sentiment -> label -> threshold sweep)
surge-train	Train all three models and produce the full evaluation report
surge-cross-val	Test whether a model trained on one subreddit transfers to the other
surge-figures	Regenerate publication figures from saved evaluation artefacts

Command	Purpose
surge-examples	Generate worked per-record prediction examples

Pipeline behaviour is controlled centrally via a `PipelineConfig` dataclass, which holds every tuneable parameter and can be overridden via configuration files. To guarantee determinism across runs, a fixed global seed (default 42) is systematically set across Python’s native random module, NumPy, and all scikit-learn estimators.

4.2 Exploratory Data Analysis Tooling

The dataset-selection decisions in Section 3.2 are backed by a separate toolset kept outside the pipeline package: three Jupyter notebooks under `eda/`, run in sequence, that import nothing from `surge_pipeline`. Each re-implements the little shared logic it needs (ticker extraction, VADER scoring) and writes a stand-alone CSV (plus figures, for the last) to `eda/output/`. Table 25 lists the three; the stages below map them onto the decisions in Section 3.2.

Table 25: EDA notebooks, in run order, with their inputs and outputs. All three are standalone and share no code with the pipeline package.

Notebook	Screening stage	Input	Output artefact
01_discovery.ipynb	Candidate discovery	Kaggle + HuggingFace dataset APIs	candidates.csv
02_highlevel_eval.ipynb	High-level comparative profiling	Shortlisted CSVs (20k-row sample each)	highlevel_comparison.csv
03_deep_assessment.ipynb	Deep viability assessment	Selected Reddit datasets (up to 100k rows)	deep_assessment.csv + figures

Stage 1: candidate discovery (`01_discovery.ipynb`). The notebook queries the Kaggle and HuggingFace dataset APIs for financial social-media data (“twitter finance”, “reddit finance”) and returns 47 raw candidates (37 Kaggle, 10 HuggingFace). Since these APIs rarely expose column schemas, completeness is inferred coarsely from titles and tags, and candidates are ranked by a score blending that with log-scaled download popularity. This coarse funnel yields a draft shortlist, not a decision. The `leukipp/reddit-finance-data` archive (Leukipp 2021) appears in it alongside several Twitter and tweet-based alternatives.

Stage 2: high-level comparative profiling (`02_highlevel_eval.ipynb`). The shortlisted datasets are profiled side by side on cheap properties: column schema, date span, per-column missingness, sampled ticker diversity, bullish/bearish ratio, and a `surge_label_ready` flag for whether the fields a surge label needs (text, timestamp, engagement) are present. Profiling reads a 20,000-row sample per dataset and writes `highlevel_comparison.csv`. The result (Table 26) settles the platform decision from Section 3.2: the two Reddit submission datasets carry engagement fields and are surge-label-ready, whereas the Twitter and tweet-based datasets carry none and cannot support a surge label whatever their ticker vocabulary.

Table 26: High-level dataset comparison from the EDA screening. Profiled on a 20,000-row sample per dataset; the Twitter-derived datasets are excluded because they carry no engagement fields and cannot support a surge label.

Dataset	Records (sampled)	Date span	Engage-ment fields	Surge-label ready
r/pennystocks submissions	20,000	2021-01-01 to 2021-02-16	Yes	Yes
WSB submissions	20,000	2021-01-01 to 2021-01-19	Yes	Yes
financial-tweets (stockerbot)	20,000	2018-02-23 to 2018-07-19	No	No
sentiment-analysis-financial-tweets	20,000	2018-02-23 to 2018-07-19	No	No

The leukipp/reddit-finance-data archive bundles several financial subreddits. A preliminary screening against the subreddit criteria from Section 3.2 narrowed these to the two most suitable: WSB (high-density) and r/pennystocks (sparse), spanning opposite ends of the posting-density spectrum to serve the abundance-versus-scarcity and cross-dataset-transfer goals; single-ticker or mostly long-form communities were unfit for the per-ticker surge design. Profiling above (Table 26) is therefore reported for this pair, with full-run sizes in Table 28.

Stage 3: deep viability assessment (03_deep_assessment.ipynb). The two surviving Reddit datasets are deep-dived on a larger sample (up to 100,000 rows) for data quality, temporal coverage and gaps, and VADER-versus-TextBlob sentiment agreement, then run through a surge-viability sweep of nine candidate definitions crossing three volume percentiles (0.90, 0.95, 0.99) with three standard-deviation multipliers (0.5, 1.0, 1.5). A dataset is suitable only when the surge-label fields exist and at least one definition puts over 2% of posts in the positive class. Both pass (Table 27): r/pennystocks with full-year coverage and stronger sentiment agreement, WSB with far higher volume in a narrower window.

Table 27: Deep viability assessment from the EDA screening. A definition is viable when over 2% of posts qualify; the “best” rate is the maximum across the sweep, at the loosest definition (percentile 0.90, multiplier 0.5). Both datasets clear the bar, confirming a workable positive class before any pipeline development.

Property	r/pennystocks	WSB
Records assessed	54,785	100,000
Date range (sampled)	2021-01-01 to 2021-12-31	2021-01-01 to 2021-01-28
Coverage / gaps (>7 days)	364 days / 0	27 days / 0
Sentiment agreement (VADER vs TextBlob)	0.723 (good)	0.661 (moderate)
Viable surge definitions (>2% positive)	4 / 9	3 / 9
Best (loosest-definition) positive rate	5.8%	5.0%
Recommendation	Suitable	Suitable

Figure 7 shows the WSB sweep and how the positive-class rate shrinks as the definition tightens, while Figure 8 sets the two communities side by side on the properties behind the sparse-versus-dense framing.

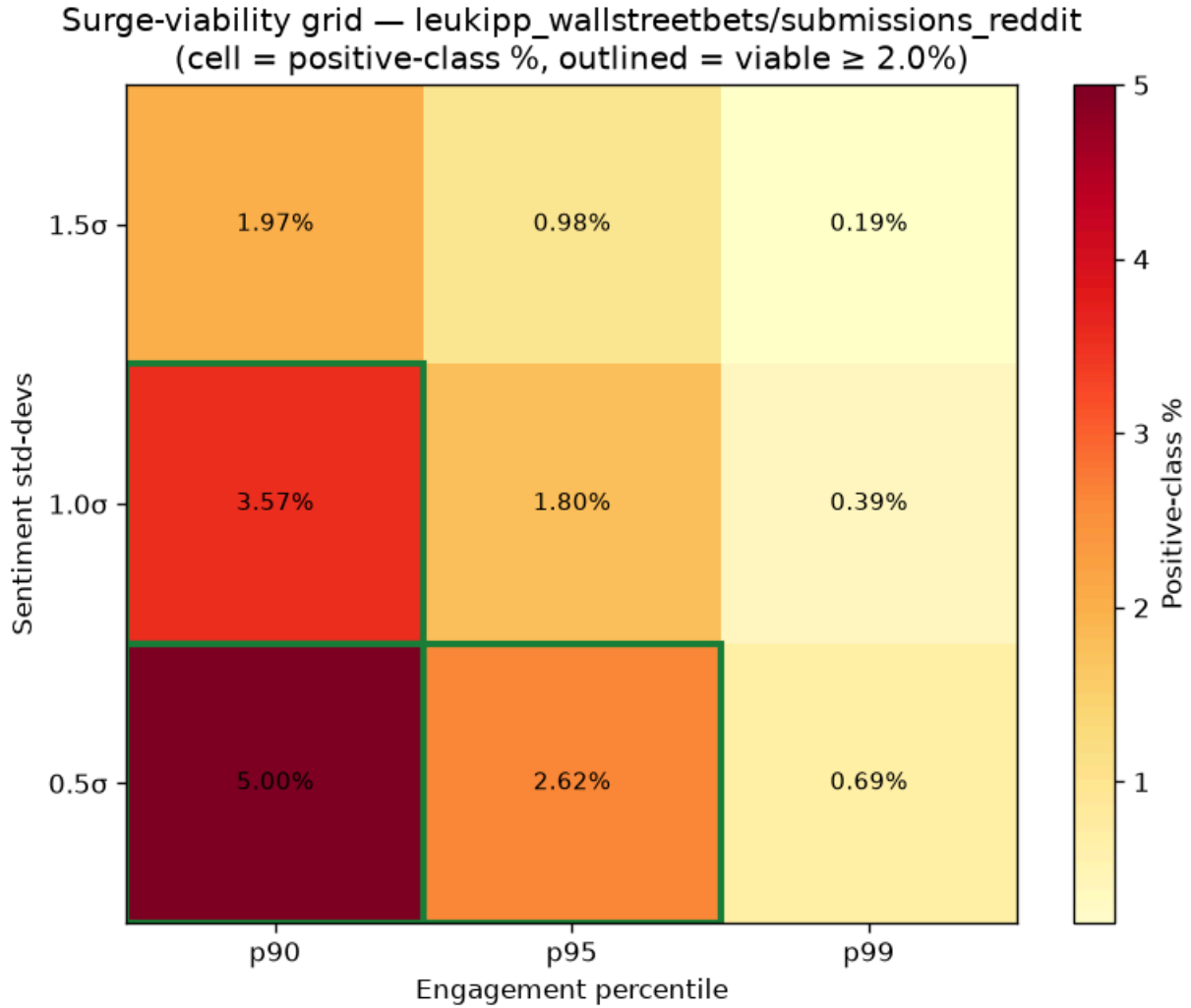


Figure 7: Surge-viability sweep for WSB from the EDA screening. Each cell reports the positive-class rate for a candidate surge definition (a volume percentile crossed with a standard-deviation multiplier). Shaded cells clear the minimum viable positive-class threshold, confirming that a usable surge signal exists before any pipeline development.

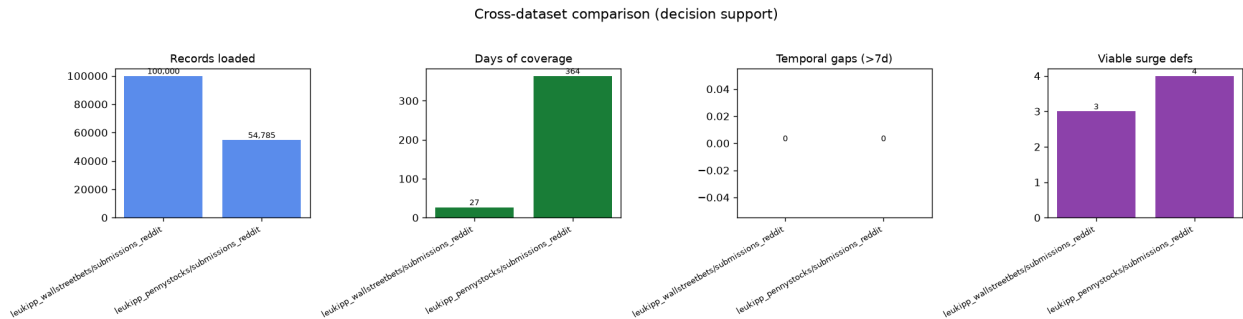


Figure 8: Cross-dataset comparison from the EDA phase, setting r/pennystocks and WSB side by side on volume, coverage, and signal properties. This exploratory contrast is what motivates the sparse-versus-dense experimental design later formalised in the evaluation.

These figures are screening artefacts, not pipeline results: the sampling caps and the deliberately simple sweep heuristic (Section 3.5) mean the EDA's counts and labels differ from the full-run pipeline's (Table 29).

4.3 Data Loading and Preprocessing

The data loader (`loader.py`) turns raw submission exports into the unit of analysis (one row per record-ticker pair, sorted chronologically) in four steps.

Step 1: Text Cleaning. Moderation placeholders (`[deleted]`, `[removed]`) and nulls in `selftext` and `title` become empty strings so the regex sees consistent input. Each row has a unique submission ID, so no deduplication is needed.

Step 2: Ticker Extraction. Two prioritised regex patterns are applied: dollar-sign tickers (`\$([A-Z]{1,5})`), e.g. `$AMC`), the highest-confidence marker, and standalone 2–5 character uppercase words (`\b[A-Z]{2,5}\b`), a broader net. Matches are filtered against a stopword lexicon that is *derived* rather than hand-coded, for transparent provenance: an **English base** taken mechanically from the NLTK stopwords corpus (upper-cased, restricted to 1–5 character tokens) is unioned with a curated, categorised **domain supplement** of finance/Reddit/market terms that resemble tickers (`DD`, `YOLO`, `NASDAQ`, `CEO`) plus everyday words NLTK omits (`HUGE`, `TECH`, `STOCK`). The supplement grew from iterative error analysis on early runs (inspect frequent uppercase false positives, categorise, repeat). A build script (`src/build_stopwords.py`) writes the lexicon to a self-documenting reference file (`input/reference/ticker_stopwords.txt`) that the loader reads at startup. A stopword filter beats a closed exchange-listed universe because tickers rotate frequently: its worst case is minor per-record noise, whereas a stale master list would silently drop unknown stocks. Listing 2 shows the logic:

Listing 2: Ticker extraction with dual regex priority cascade and stopword filtering (from `loader.py`). The dollar-sign pattern captures explicit financial references with high precision; the uppercase pattern broadens recall at the cost of precision, mitigated by a stopword lexicon derived from an NLTK English base plus a curated domain supplement (see ``src/build_stopwords.py``).

```
# Extract tickers from combined title + selftext
combined_text = f"{title!s} {selftext!s}"

# 1. Dollar-sign pattern (highest priority – always included)
dollar_matches = _DOLLAR_SIGN_PATTERN.findall(combined_text)
for match in dollar_matches:
    ticker = match.upper()
    if ticker not in TICKER_STOPWORDS:
        tickers.add(ticker)

# 2. Uppercase word pattern (filtered against stopwords)
word_matches = _UPPERCASE_WORD_PATTERN.findall(combined_text)
for match in word_matches:
    ticker = match.upper()
    if ticker not in TICKER_STOPWORDS:
        tickers.add(ticker)
```

Step 3: Timestamp Normalisation. Both archive formats (Unix epoch integers and ISO datetime strings) are unified into timezone-aware `datetime64[ns, UTC]` and sorted chronologically, a hard precondition for the binary-search windowing that follows. A missing timestamp column raises an explicit error rather than failing silently.

Step 4: Ticker Explosion & Filtering. Multi-ticker posts are split and exploded via `pandas.explode()` into one row per record–ticker pair (Section 3.4); records with zero valid tickers are dropped.

Table 28: Loader-stage attrition.

Step	r/pennystocks	WSB
Raw records loaded	304,524	1,293,981
Excluded (no tickers found)	224,312 (73.7%)	716,109 (55.3%)
After explosion (record-ticker pairs)	80,212	577,872

After the remaining stages (labelling also drops records whose forward window is too sparse or runs past the dataset boundary, Section 3.5), the datasets resolve to the characteristics in Table 29. These full-run figures, not the capped EDA samples (Section 4.2), are used throughout the evaluation.

Table 29: Dataset characteristics. Surge counts and imbalance ratios are at the primary threshold ($\tau = 1.5$, $w_1 = w_2 = 0.5$).

Property	r/pennystocks	WSB
Raw records	304,524	1,293,981
Date range	2021-01-01 to 2021-12-31	2021-01-01 to 2021-12-31
After ticker extraction (exploded)	80,212	577,872
Usable records (post-exclusion)	24,827	457,072
Train / Test split	21,549 / 3,278	388,149 / 68,923
Train surges	667	5,934
Test surges	31	668
Train imbalance ratio	31:1	64:1
Test imbalance ratio	105:1	102:1

4.4 Feature Engineering (Implementation)

The eleven design features (Table 15) are computed here under one constraint: each must be derivable from information available *at or before* the record’s timestamp t , never from the forward window that builds the label. Content features (`sentiment_score`, `word_count`, `title_length`, `num_tickers_mentioned`) read only the record’s own text and temporal features (`hour_of_day`, `day_of_week`) come straight from `created_utc`, so both are trivially backward-safe. The activity features and their interaction terms are where the constraint bites: they count and compare *prior* same-ticker posts, which the windowing and boundary logic below enforces. Post-hoc engagement metrics (`Reddit score`, `num_comments`) are excluded entirely, as verified by Listing 10.

Table 30: How each feature is produced and computed. Definitions and categories are given in the design (Table 15); this table records the implementation.

#	Feature	Produced by	Computation
1	ticker_post_rate_24h	windowing.compute_windowed_counts() (reused)	np.searchsorted count over $(t - 24 \text{ h}, t)$; side='left' at t excludes the self-post
2	time_since_previous	features._compute_time_since_previous()	Per-ticker np.diff(times); -1 for a ticker's first occurrence
3	ticker_post_acceleration	features._compute_ticker_post_acceleration()	Recent/older 12h split counted by binary search (see below)
4	sentiment_score	sentiment.compute_sentiment() (reused)	VADER on own text only, title-fallback when self-text empty
5	word_count	features.compute_features() inline	Token count of own title + selftext
6	title_length	features.compute_features() inline	Token count of own title
7	num_tickers_mentioned	features._compute_num_tickers_mentioned()	groupby('id') ['ticker'].transform('nunique') on the record's own post
8	hour_of_day	features.compute_features() inline	created_utc.dt.hour
9	day_of_week	features.compute_features() inline	created_utc.dt.dayofweek
10	word_count_x_hour	features.compute_features() inline	Element-wise product of word_count and hour_of_day
11	accel_x_time_since_prev	features.compute_features() inline	Element-wise product of ticker_post_acceleration and time_since_previous (clamped to 0)

Three points are worth drawing out; the remaining features are direct column operations.

Acceleration is the most involved computation. ticker_post_acceleration splits the backward 24-hour window into a recent half $(t - 12 \text{ h}, t)$ and an older half $(t - 24 \text{ h}, t - 12 \text{ h}]$ and takes the ratio of their post counts, so a value above 1.0 marks accelerating discussion. On pre-sorted per-ticker timestamp arrays, four np.searchsorted calls count both halves in $O(n \log n)$ per ticker group, avoiding a per-record loop:

Listing 3: Ticker post acceleration via split-window binary search (from features.py). The backward 24-hour window is bisected into recent and older halves. Four searchsorted calls per ticker group compute counts in each half; the ratio detects whether posting is accelerating (>1.0) or decelerating (<1.0). The $\max(\dots, 1)$ guard prevents division by zero when the older half is empty.

```

# Count posts in recent half (t-12h, t) excluding self
recent_left = np.searchsorted(times, times - _12H_SECONDS, side="right")
recent_right = np.searchsorted(times, times, side="left")
count_recent = recent_right - recent_left

# Count posts in older half (t-24h, t-12h]
older_left = np.searchsorted(times, times - _24H_SECONDS, side="right")
older_right = np.searchsorted(times, times - _12H_SECONDS, side="right")
count_older = older_right - older_left

acceleration = count_recent / np.maximum(count_older, 1)

```

Boundary conventions make the backward-only guarantee exact. `side='left'` at the right edge (t) excludes the record's own post, so a feature never sees the event it predicts; `side='right'` at the left edge gives an exclusive-left boundary. The `max(count_older, 1)` guard prevents division by zero when the older half is empty, common for a newly discussed ticker. The same count, computed once in `windowing.py`, is reused for `ticker_post_rate_24h` rather than recomputed, keeping a single source of truth.

Interaction terms are a deliberate design choice. `word_count_x_hour` and `accel_x_time_since_prev` are explicit products (long analytical posts at peak trading hours; sudden acceleration after silence) given to the models rather than left for deep tree splits to reconstruct. Since `time_since_previous` is -1 for a ticker's first occurrence, it is clamped to 0 before forming `accel_x_time_since_prev` so a missing history never becomes a spurious negative product. An ablation confirmed a consistent +1.4pp AUC lift from both terms.

4.5 Surge Labelling

The labelling module turns windowing and sentiment outputs into binary surge labels under strict temporal isolation.

Temporal Split. Records are split chronologically at the 80th-percentile timestamp (no shuffling): those at or before the cutpoint go to training, the rest to test. Ties are resolved to training, the leakage-safe choice when many per-second timestamps share the split instant, so the realised train fraction can drift slightly from 0.80.

Z-score Normalisation. The mean (μ) and standard deviation (σ) of both the volume growth ratio and the sentiment shift are computed from training records only, then applied unchanged to both partitions:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Standardising the test set against training-derived distributions prevents future data leaking into historical baselines.

Composite Metric and Thresholding. The composite combines the standardised metrics:

$$\text{composite} = (w_{\text{volume}} \cdot z_{\text{volume}}) + (w_{\text{sentiment}} \cdot z_{\text{sentiment}})$$

where $z_{\text{sentiment}}$ standardises the *absolute* shift $|\Delta S|$ (Section 3.5), so a surge tracks how much sentiment moves, not its direction. A record is labelled a surge ($y = 1$) when its composite exceeds threshold τ , else non-surge ($y = 0$).

Listing 4: Surge labelling pipeline (from `labelling.py`). Steps 1-2 establish the leakage-prevention mechanism: mean and standard deviation are estimated exclusively from the training partition, then applied unchanged to all records including the test set. Steps 3-4 combine the normalised volume growth and

sentiment shift into a weighted composite and threshold it into a binary target. Because test-set records are normalised against a distribution they never contributed to, the target labels encode no future information. Population standard deviation (ddof=0) is used because the training partition constitutes the entire reference population for normalisation, not a sample drawn from a larger one. Normalisation is delegated to a helper (`_compute_z_scores`) that returns zeros when a standard deviation is zero, and excluded records are marked NaN rather than dropped.

```
# Step 1: Temporal split at 80th percentile timestamp
epoch_seconds = to_epoch_seconds(df["created_utc"])
split_ts = np.percentile(epoch_seconds, ratio * 100)
partitions = np.where(epoch_seconds <= split_ts, "train", "test")

# Step 2: Compute  $\mu/\sigma$  from training partition ONLY
train_volume = df.loc[train_mask, "posting_volume_growth"].values.astype(np.float64)
train_sentiment = np.abs(
    df.loc[train_mask, "sentiment_change"].values.astype(np.float64)
)
mu_vol = float(np.mean(train_volume))
sigma_vol = float(np.std(train_volume, ddof=0)) # population std
mu_sent = float(np.mean(train_sentiment))
sigma_sent = float(np.std(train_sentiment, ddof=0))

# Step 3: Z-score normalise ALL records using frozen training stats.
# _compute_z_scores() guards the  $\sigma=0$  edge case by returning zeros.
z_volume = _compute_z_scores(all_volume, mu_vol, sigma_vol)
z_sentiment = _compute_z_scores(all_sentiment, mu_sent, sigma_sent)

# Step 4: Composite metric and binary labelling
composite = (w1 * z_volume) + (w2 * z_sentiment)
surge_label = np.where(composite > tau, 1.0, 0.0)
```

A record is excluded as unlabellable when its forward window holds fewer than two same-ticker posts (avoiding meaningless growth ratios) or extends past the dataset’s final timestamp. Excluded records are not dropped: their z-scores, composite, and label are set to NaN and skipped in class-distribution tallies, so exclusion never adds a spurious non-surge. These rules drive the attrition from 577,872 exploded records to 457,072 usable ones on WSB (Table 29). For sensitivity analysis, `sweep_thresholds()` evaluates $\tau \in \{0.5, 1.0, 1.5, 2.0, 2.5\}$ in one vectorised pass; setting $w_{\text{sentiment}} = 0$ gives the volume-only Phase 1 variant.

4.6 Model Training

Training tunes each model family on past-only validation, then locks a decision threshold before the test set is touched, following the develop $\rightarrow$ tune $\rightarrow$ retrain $\rightarrow$ test-once workflow of Figure 6. The blocks below cover cross-validation, class-imbalance handling, and the retrain-and-lock step.

Expanding-window Cross-validation

The training partition is split into four chronological blocks, giving three expanding validation splits (Table 31).

Table 31: Expanding-window validation splits.

Split	Train	Validate
1	Block 1	Block 2

Split	Train	Validate
2	Blocks 1–2	Block 3
3	Blocks 1–3	Block 4

Every split enforces $\max(t_{\text{train}}) < \min(t_{\text{val}})$ to prevent lookahead bias (Listing 5):

Listing 5: Expanding-window split construction and temporal verification (from training.py). The expanding window concatenates all preceding folds as training data, validating on the immediately subsequent fold. The hard assertion $\max_{\text{train}} > \min_{\text{val}}$ triggers a `ValueError` if any split violates chronological ordering, making lookahead leakage a crash rather than a silent corruption.

```
def get_expanding_window_splits(folds):
    """Convert sequential folds into expanding-window train/val splits."""
    splits = []
    for i in range(1, len(folds)):
        train_idx = np.concatenate(folds[:i])
        val_idx = folds[i]
        splits.append((train_idx, val_idx))
    return splits

def _verify_temporal_ordering(df, folds, splits):
    """Verify that all splits respect chronological ordering."""
    epoch = to_epoch_seconds(df["created_utc"])
    for i, (train_idx, val_idx) in enumerate(splits):
        max_train = epoch[train_idx].max()
        min_val = epoch[val_idx].min()
        if max_train > min_val:
            raise ValueError(
                f"Temporal ordering violated in split {i}: "
                f"max(train_ts)={max_train} > min(val_ts)={min_val}"
            )
```

Hyperparameters are tuned by grid search per model class (Table 32).

Table 32: Hyperparameter search spaces.

Model	Parameters Searched	Grid Size
Logistic Regression	$C \in \{0.01, 0.1, 1, 10, 100\}$, $\text{l1_ratio} \in \{0, 1\}$	10
Random Forest	$n_{\text{estimators}} \in \{50, 100, 200\}$, $\text{max_depth} \in \{3, 5, 10, \text{None}\}$, $\text{min_samples_leaf} \in \{1, 2, 5\}$	36
XGBoost	$n_{\text{estimators}} \in \{50, 100, 200\}$, $\text{max_depth} \in \{3, 5, 7\}$, $\text{learning_rate} \in \{0.01, 0.1, 0.3\}$, $\text{scale_pos_weight} \in \{1, \text{ratio}/2, \text{ratio}\}$	≤ 50

The XGBoost grid ($3 \times 3 \times 3 \times 2 - 3 = 54$ –81 configs, fewer when $\text{ratio}/2$ collides with 1) is truncated to 50 to bound search time.

A `StandardScaler` is fitted on each split’s training fold only, then applied to its validation fold, preventing leakage (Listing 6):

Listing 6: Grid search with per-fold scaler isolation (from training.py). Each fold fits a fresh StandardScaler on training indices only, then transforms the validation fold using those frozen statistics. This prevents mean/variance leakage across the temporal boundary. The best configuration is retrained on the entire training partition before test-set evaluation, maximising the data available to the final model.

```
for params in param_grid:
    fold_aucs = []
    for train_idx, val_idx in splits:
        # Fit scaler on training fold ONLY, prevents validation leakage
        scaler = StandardScaler()
        X_fold_train = scaler.fit_transform(X_train_full[train_idx])
        X_fold_val = scaler.transform(X_train_full[val_idx])
        y_fold_train = y_train_full[train_idx]
        y_fold_val = y_train_full[val_idx]

        # Under severe imbalance a fold may hold no positives; AUC is
        # then undefined, so score it as chance (0.5) and skip.
        if len(np.unique(y_fold_val)) < 2:
            fold_aucs.append(0.5)
            continue

        model = make_model_fn(params, random_seed)
        model.fit(X_fold_train, y_fold_train)

        y_prob = model.predict_proba(X_fold_val)[: , 1]
        auc = roc_auc_score(y_fold_val, y_prob)
        fold_aucs.append(auc)

    mean_auc = np.mean(fold_aucs)
    if mean_auc > best_mean_auc:
        best_mean_auc = mean_auc
        best_params = params

# Retrain winner on full training partition
final_scaler = StandardScaler()
X_train_scaled = final_scaler.fit_transform(X_train_full)
final_model = make_model_fn(best_params, random_seed)
final_model.fit(X_train_scaled, y_train_full)
```

Model Instantiation and Class Imbalance Handling

A factory function per model injects the class-imbalance strategy into training (Listing 7):

Listing 7: Model factory functions (from training.py). All three models handle class imbalance through cost-sensitive learning rather than synthetic oversampling. Logistic Regression and Random Forest use class_weight balanced (sklearn automatically computes inverse frequency weights). XGBoost uses scale_pos_weight, grid-searched over {1, ratio/2, ratio} where ratio = n_negative / n_positive (typically 19:1 to 105:1 in this dataset). This avoids SMOTE incompatibility with temporal data, where synthetic records lack meaningful timestamps.

```
def _make_lr(params, random_seed):
    return LogisticRegression(
        C=params["C"],
```

```

        ll_ratio=params["ll_ratio"],
        solver="saga",          # supports Elastic Net (L1 + L2)
        class_weight="balanced", # inversely weight classes by frequency
        random_state=random_seed,
        max_iter=5000,
    )

def _make_rf(params, random_seed):
    return RandomForestClassifier(
        n_estimators=params["n_estimators"],
        max_depth=params["max_depth"],
        min_samples_leaf=params["min_samples_leaf"],
        class_weight="balanced", # per-tree class reweighting
        random_state=random_seed,
        n_jobs=-1,
    )

def _make_xgb(params, random_seed):
    return XGBClassifier(
        n_estimators=params["n_estimators"],
        max_depth=params["max_depth"],
        learning_rate=params["learning_rate"],
        scale_pos_weight=params["scale_pos_weight"], # neg/pos ratio
        random_state=random_seed,
        eval_metric="logloss",
        verbosity=0,
    )

```

That factory's `scale_pos_weight` grid is anchored on the base ratio r from the training partition's class distribution:

$$r = \frac{N_{\text{negative}}}{N_{\text{positive}}}$$

Listing 8: Dynamic imbalance ratio computation (from `training.py`). The negative-to-positive ratio is calculated from the actual training partition class distribution, then used to construct a three-level grid for XGBoost `scale_pos_weight`: no reweighting (1.0), moderate ($\text{ratio}/2$), and full (ratio). This data-driven approach adapts automatically to different surge thresholds and datasets without manual tuning.

```

n_positive = int(np.sum(y_train_full == 1))
n_negative = int(np.sum(y_train_full == 0))
imbalance_ratio = float(n_negative) / max(n_positive, 1)

# Grid searches over: no reweighting, moderate, and full reweighting
weight_values = sorted(set([1.0, imbalance_ratio / 2, imbalance_ratio]))

```

The configuration with the highest mean validation AUC across the three splits (Section 3.8) wins, and is retrained on the full 80% training partition with a freshly fitted `StandardScaler`. The operating point is fixed here too: the retrained model scores the last validation fold to pick an F_1 -maximising threshold (Section 4.7), applied unchanged to the test set. Each model is serialised to a versioned `.joblib` (estimator, scaler, best parameters, threshold) with a `latest_models.json` manifest, so evaluation and cross-dataset transfer load a run without retraining.

4.7 Evaluation Pipeline Implementation

So that every reported number and plot derives from a single re-runnable path rather than ad-hoc analysis, evaluation is implemented as a reusable set of functions (`evaluation.py`), figure generators (`evaluation_figures.py`), and an append-only run tracker (`experiment_log.py`).

Metric computation. For each trained model, held-out predictions are scored for AUC-ROC (the primary metric), plus precision, recall, and F_1 at both the default 0.5 threshold and the validation-tuned threshold (fixed during training, Section 3.8). Uncertainty is quantified by 1,000 bootstrap resamples of the test set, yielding 95% confidence intervals for every metric, and pairwise model differences are tested with McNemar’s test under a Bonferroni-corrected α .

Experiment orchestration. A run is fully specified by a `PipelineConfig` (dataset, weights w_1/w_2 , threshold τ , seed). The CLI entry points (Table 24) execute a run end-to-end; each writes its resolved configuration, metrics, and environment to a timestamped directory and appends one line to a JSONL log recording the config, the Git commit SHA, and the output path. Because a run is reproducible from its config and seed, model comparisons, threshold sweeps (τ), and sentiment-weight sweeps (w_2) are executed by re-running with different configs rather than by editing code, and the cross-dataset transfer experiment simply loads a model trained on one community and scores it on the other’s test set.

Artefacts and figures. Each run emits machine-readable JSON (metrics, confidence intervals, per-model confusion counts) alongside the figures used in Section 5: ROC curves, confusion matrices, threshold-sensitivity and feature-importance plots, regenerated deterministically from the saved evaluation JSON by `surge-figures`. Separating figure generation from metric computation means the reported tables and plots cannot diverge: both read the same artefacts.

4.8 Implementation Decisions Driven by Empirical Findings

Iterative development surfaced three failures serious enough to reshape the design; each is recorded here with the diagnosis and the safeguard it produced:

Timestamp Unit Mismatch & Temporal Integrity

A timestamp-unit conversion error during temporal splitting silently excluded 89% of records, leaving just 7 positive surges in the test set. Beyond fixing the bug, the scare motivated automated assertions, such as $\max(t_{\text{train}}) < \min(t_{\text{val}})$, baked into the training pipeline (Listing 5) so any future violation crashes rather than corrupts.

Data Sparsity and Dataset Scaling

Early `r/pennystocks` runs (~80,000 records) yielded as few as 7 positive test examples at stricter thresholds ($\tau \geq 2.0$), leaving AUC estimates at the mercy of noise. Scaling up to `WSB` (577,872 records, 668 test surges at $\tau = 1.5$) stabilised the metrics and made the cross-dataset transfer experiments viable.

Class Imbalance and Decision Boundary Calibration

At $\tau = 1.5$, extreme imbalance (1.44% surge rate; 102:1) led an early XGBoost run to a strong AUC of 0.888 (near the final 0.892) yet zero positive predictions at the 0.5 threshold, a calibration issue, not a model failure. Lowering τ to 1.0 (~5% surge rate) and adding `scale_pos_weight` to the search grid (Listing 8) restored both probability alignment and recall.

4.9 Implementation Status

All six pipeline stages (Table 11) execute end-to-end on both datasets to produce reproducible artefacts, directly serving goal G5. End-to-end runtime is about 8 minutes for `r/pennystocks` and 19 minutes for `WSB` on a laptop CPU, most of it VADER sentiment computation.

Determinism was verified empirically: re-running the `r/pennystocks` baseline (configuration A1, Table 21, seed 42) on two dates produced identical AUC (0.753) and byte-identical logs. Results are robust across the five robustness seeds on `r/pennystocks`, with AUC spanning 0.734 to 0.753 (a 0.019 margin). All 30+ runs in the matrix (Table 21) log their configuration JSON, Git commit SHA, and output path, so any reported result traces back to the exact configuration and code revision that produced it.

4.10 Testing Strategy

Correctness and correctness claims are maintained through a 10-module `pytest` suite, strict type checking (`mypy`), and linting (`ruff`), all passing with zero errors. The suite mirrors the library’s module structure, using hand-computed edge cases to verify mathematical and temporal invariants per stage.

Table 33: Test modules and the invariants they verify.

Test Module	Pipeline Stage	Key Invariants Tested
<code>test_loader.py</code>	Data Loading	Ticker regex extracts known patterns; stop-word filter blocks false positives; explosion produces correct row count
<code>test_windowing.py</code>	Temporal Windowing	Forward/backward counts match hand-computed values; boundary inclusion/exclusion semantics; per-ticker independence
<code>test_sentiment.py</code>	Sentiment	VADER scores match reference; title-fall-back activates when selftext is empty
<code>test_labelling.py</code>	Target Labelling	Z-scores use training-only μ/σ ; composite formula correctness; threshold labelling at known values
<code>test_features.py</code>	Feature Engineering	No feature accesses future data; <code>score/num_comments</code> excluded; <code>time_since_previous</code> is backward-only
<code>test_training.py</code>	Model Training	Temporal folds are chronologically ordered; expanding-window splits respect $\max(\text{train}) < \min(\text{val})$; grid sizes ≤ 50
<code>test_evaluation_significance.py</code>	Evaluation	McNemar’s test produces correct χ^2 for known contingency tables; bootstrap CIs have expected coverage
<code>test_config.py</code>	Configuration	JSON serialisation round-trip preserves all parameters
<code>test_pipeline_integration.py</code>	End-to-end	Full pipeline produces identical output across two runs (determinism)
<code>test_evaluation_figures.py</code>	Figures	Figure generation completes without error on synthetic data

Temporal Leakage Prevention

The most critical test module (`test_labelling.py`) verifies that normalisation statistics do not leak future information into historical records. The test constructs synthetic data with distinct training ($\mu = 5.0, \sigma = 5.0$) and test ($\mu = 17.5, \sigma = 2.5$) distributions, then asserts that test-set z -scores are derived exclusively using the training parameters:

Listing 9: Leakage-prevention test (from `test_labelling.py`). The test constructs data with known training and test distributions, then asserts that test-set z -scores are computed using training parameters (mean=5, sd=5) rather than test-set parameters (mean=17.5, sd=2.5). A negative assertion confirms the wrong computation does not occur.

```
class TestZScoreNormalisation:
    def test_test_set_uses_training_stats_not_own(self, base_timestamp):
        """Critical: test-set records are normalised with TRAINING statistics."""
        # Training [0..7]: values = [0,0,0,0,10,10,10,10] ->  $\mu=5, \sigma=5$ 
        # Test [8..9]: values = [15, 20]
        volumes = [0,0,0,0, 10,10,10,10, 15, 20]
        ...
        # Multi-ticker explosion expands these into more rows, so the
        # test-set records land at iloc 11 and 15 in the labelled frame.
        result = apply_labelling(df, config)

        # Test z-scores MUST use training stats:  $z(15)=(15-5)/5=2.0$ 
        assert result.df["z_volume"].iloc[11] == approx(2.0)
        assert result.df["z_volume"].iloc[15] == approx(3.0)

        # If they wrongly used test-only stats ( $\mu=17.5, \sigma=2.5$ ):
        #  $z(15)$  would be -1.0, verify this is NOT the case
        assert result.df["z_volume"].iloc[11] != approx(-1.0)
```

Feature Time-Boundary Contracts

A second critical test suite (`test_features.py`) enforces strict backward-only feature calculation. It programmatically validates that: - Inter-arrival features (e.g., `time_since_previous`) depend strictly on preceding timestamps $t' \leq t$. - Post-hoc engagement signals (e.g., upvotes or comments accumulated after publication time t) are structurally excluded from the feature design matrix.

Listing 10: Engagement-exclusion contract test (from `test_features.py`). The test asserts that post-hoc engagement metrics, which accumulate after publication time t , are structurally excluded from the feature set. The backward-only inter-arrival check is verified by a companion test in the same suite.

```
class TestNoFutureLeakage:
    def test_features_exclude_score_and_num_comments(self, labelled_df):
        """Post-hoc engagement metrics must NOT appear in features."""
        assert "score" not in FEATURE_COLUMNS
        assert "num_comments" not in FEATURE_COLUMNS
```

These unit tests execute on synthetic datasets and run automatically prior to every experiment. They provide continuous assurance that refactoring or parameter changes cannot silently introduce temporal leakage.

5 Evaluation (2170/2500 words)

5.1 Evaluation Against Project Objectives

The three objectives (Table 2) were each assessed against their success test on the held-out data. Table 34 records the verdict and points to the evidence that supports it, developed in the Results and Critical Analysis that follow; the detailed figures are not restated here.

Table 34: Verdicts against the three project objectives, with pointers to supporting evidence.

#	Objective	Verdict	Key evidence
O1	Predict surges	Met	Both tree models clear the stretch tier on WSB and target on r/pennystocks, well above the best single feature; the binding constraint is data density, not methodology (Table 35, Table 39)
O2	Compare approaches	Met	All pairwise model differences are significant; complexity helps under abundance but inverts under scarcity (Section 5.2.3, Section 5.3.1)
O3	Temporal validity	Met	Reported on a chronological holdout with leakage prevented at every stage; the large validation-to-test F1 gap shows the protocol exposing what random splits would have hidden (Section 5.3.6)

5.2 Results

All performance metrics are reported on the chronologically held-out test partition (the final 20% of data), which was isolated from training and threshold selection.

5.2.1 Dataset and Target Analysis

Before the model results, it is worth stating precisely what prediction problem the final labelling produced, because its difficulty bounds everything that follows. At the primary threshold ($\tau = 1.5$, $w_1 = w_2 = 0.5$), the composite target is severely imbalanced: WSB yields 668 surges in 68,923 test records (0.97%, a 102:1 negative-to-positive ratio) and r/pennystocks 31 surges in 3,278 records (0.95%, 105:1) (Table 29). Both communities therefore pose a sub-1% positive-rate problem, where a trivial “never surge” classifier scores 99% accuracy, so accuracy is uninformative and ranking metrics are essential.

Two properties make the problem genuinely hard rather than merely imbalanced. First, the positive count on r/pennystocks (31 test surges) is small enough that metric estimates carry wide uncertainty, a limitation revisited in Section 5.3.1 and the limitations. Second, the target is defined chronologically, so the test surges fall in the later part of 2021, after the post-GameStop peak; if surge dynamics shifted over the year, the test problem is not identically distributed to training, which is exactly the distribution-shift risk examined in Section 5.3.6. The label counts are consistent with the EDA viability screening (Section 4.2), which found comparable positive-class rates on sampled data, confirming the full-run labels are neither degenerate nor trivially separable.

5.2.2 Model Performance

On WSB, XGBoost achieved AUC-ROC 0.892 [0.881–0.902] and Random Forest 0.880 [0.869–0.890], both clearing the stretch tier. On r/pennystocks, Random Forest led at 0.753 [0.673–0.824], meeting target.

Table 35: Test-set performance at default classification threshold (0.5). 95% bootstrap CIs from 1,000 resamples. Tier reflects AUC-ROC only (Table 19), which is why XGBoost on r/pennystocks is still “Target” (AUC 0.734) despite 0.000 precision, recall, and F1 at this threshold.

Dataset	Model	AUC-ROC [95% CI]	Precision	Recall	F1	Tier
WSB (68,923 records, 668 surges, 0.97% rate)	LR	0.707 [0.684–0.729]	0.013	0.801	0.026	Target
	RF	0.880 [0.869–0.890]	0.095	0.311	0.145	Stretch
	XGB	0.892 [0.881–0.902]	0.043	0.819	0.081	Stretch
pennystocks (3,278 records, 31 surges, 0.95% rate)	LR	0.680 [0.588–0.778]	0.013	0.645	0.025	Minimum
	RF	0.753 [0.673–0.824]	0.068	0.194	0.101	Target
	XGB	0.734 [0.641–0.821]	0.000	0.000	0.000	Target

AUC scores are strong but precision is near zero everywhere at the default 0.5 threshold. With sub-1% surge rates, probability outputs cluster far below 0.5, models rank surges correctly but the threshold is too conservative. This is a calibration problem, not a discrimination failure. On pennystocks, RF and XGB confidence intervals overlap substantially; McNemar’s test (Section 5.2.3) nonetheless confirms significant prediction differences.

Decision thresholds were optimised on the last validation fold via F1-maximisation:

Table 36: Metrics at validation-tuned thresholds.

Dataset	Model	Tuned Threshold	Precision	Recall	F1	F1 Δ
WSB	LR	0.81	0.058	0.280	0.097	+0.071
	RF	0.88	0.180	0.051	0.079	−0.066
	XGB	0.85	0.217	0.235	0.226	+0.145
pennys-tocks	LR	0.67	0.085	0.194	0.118	+0.093
	RF	0.79	0.200	0.097	0.130	+0.030
	XGB	0.16	0.114	0.129	0.121	+0.121

After tuning, XGBoost on WSB achieves the best $F_1 = 0.226$ at threshold 0.85. Notably, XGBoost on pennystocks requires threshold 0.16 to produce any positive predictions, while RF on WSB selects 0.88 that sacrifices recall for precision (net $\Delta F_1 = -0.066$).

Table 37: Confusion matrix for the best model at tuned threshold.

Dataset	Model	Threshold	TP	FP	FN	TN
WSB	XGBoost	0.85	157	565	511	67,690

Dataset	Model	Threshold	TP	FP	FN	TN
pennystocks	Random Forest	0.79	3	12	28	3,235

At the best operating point on WSB, XGBoost identifies 157 of 668 surges with 565 false alarms (~1 true positive per 4.6 flags), manageable for human review but unsuitable for automation.

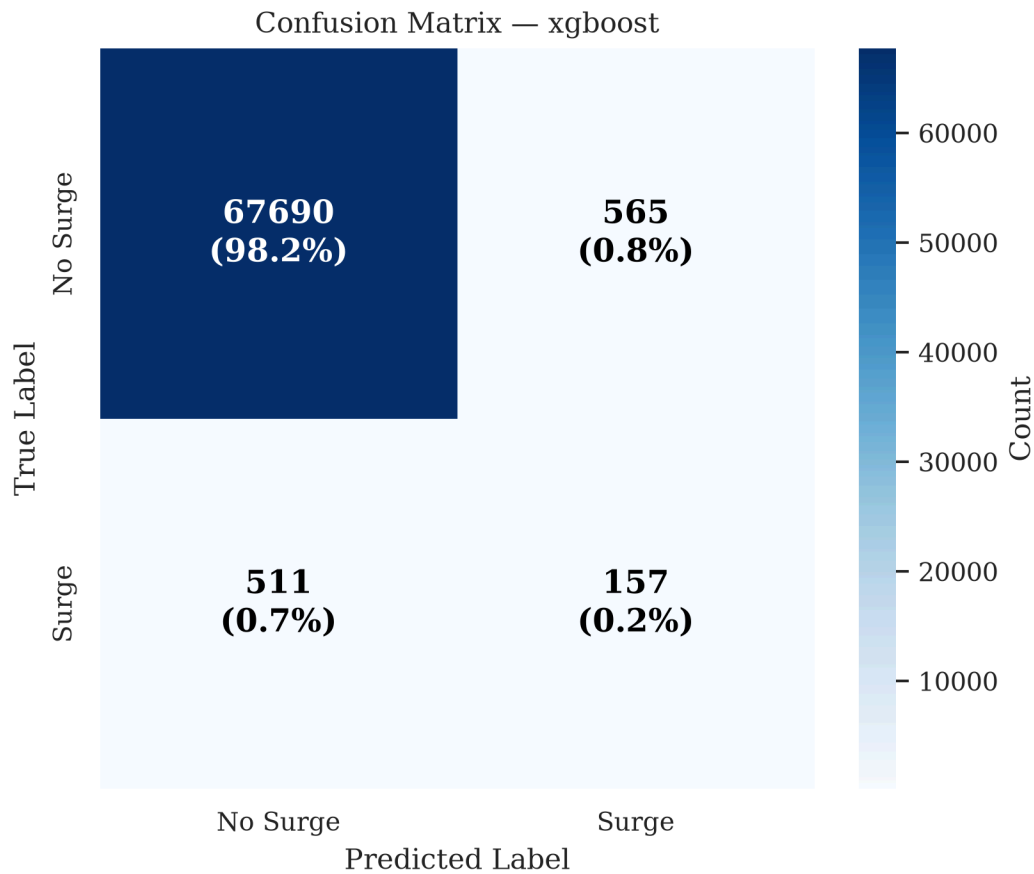


Figure 9: Confusion matrix for XGBoost at tuned threshold (0.85) on the WSB held-out test set. The model correctly identifies 157 surges (TP) while generating 565 false alarms (FP), with 511 missed surges (FN). The extreme class imbalance (67,690 TN) visually confirms why precision remains low despite strong ranking ability.

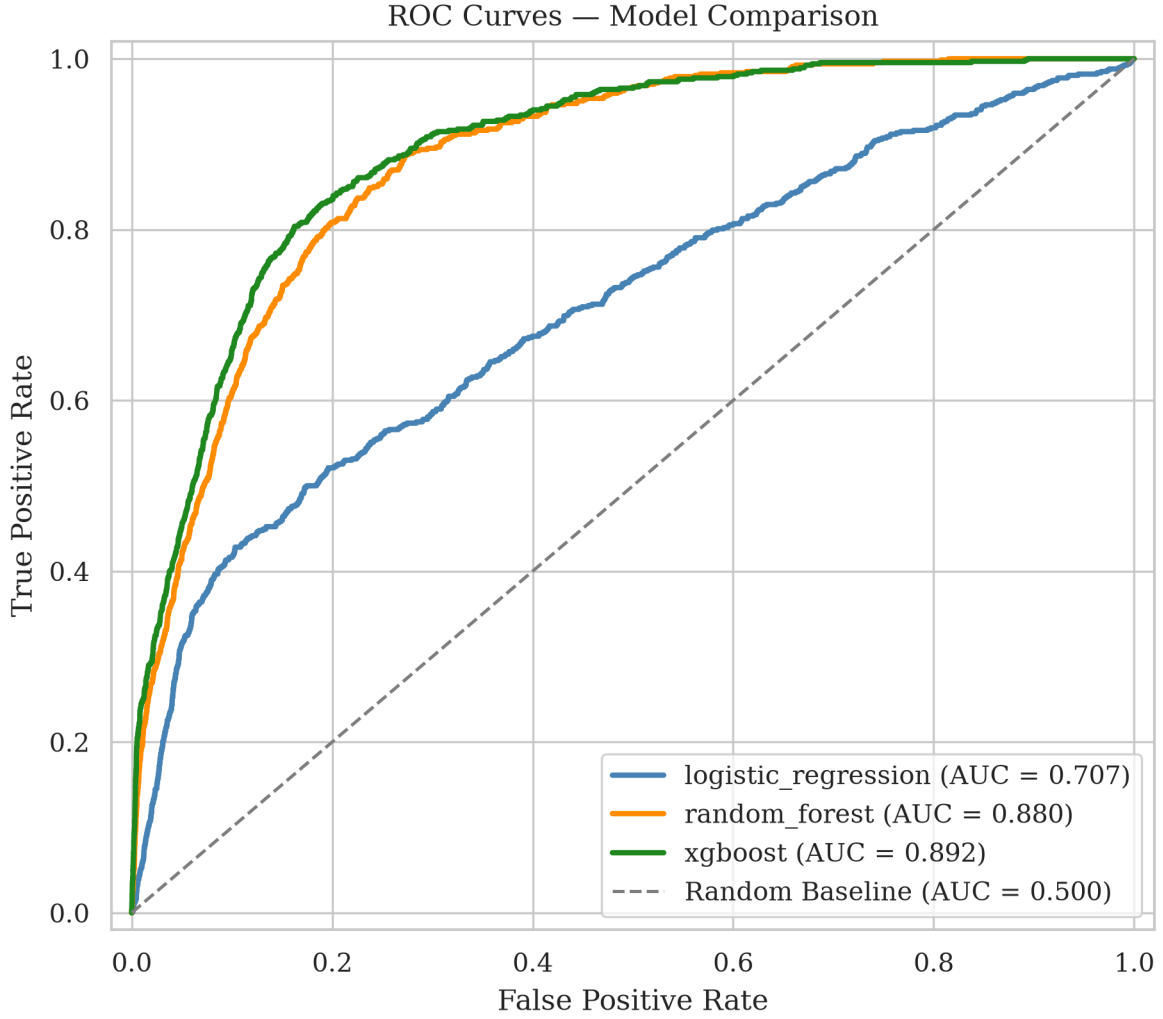


Figure 10: Combined ROC curves for Logistic Regression, Random Forest, and XGBoost on the WSB held-out test set. The diagonal represents a random classifier (AUC = 0.5).

5.2.3 Statistical Validation

All pairwise comparisons show statistically significant differences ($p < 0.001$).

Table 38: McNemar’s pairwise significance tests (default 0.5 threshold, Bonferroni-adjusted $\alpha = 0.017$).

Dataset	Model Pair	χ^2	p-value	Significant?
WSB	LR vs RF	36,664	< 0.001	Yes
WSB	LR vs XGB	24,246	< 0.001	Yes
WSB	RF vs XGB	9,208	< 0.001	Yes
pennystocks	LR vs RF	1,406	< 0.001	Yes
pennystocks	LR vs XGB	1,486	< 0.001	Yes
pennystocks	RF vs XGB	66	< 0.001	Yes

The large χ^2 values on the WSB test partition reflect both the substantial sample size ($N = 68,923$) and distinct error profiles across models, such as XGBoost predicting strictly negative instances at the 0.5 threshold while Random Forest makes selective positive predictions.

To evaluate the utility of combining multiple signals, model performance was benchmarked against the single strongest predictive feature (Section 3.1.3). On WSB, the top single-feature heuristic achieves an AUC of 0.805; combining features in the full models yields a 0.087 gain in AUC, demonstrating that multi-feature integration successfully captures complex signal interactions.

Table 39: Multi-feature models vs. baselines (AUC-ROC).

Dataset	Random Baseline	Best Single Feature	Best Model	Δ over Single Feature
WSB	0.500	0.805 (word_count)	0.892 (XGB)	+0.087
pennys-tocks	0.500	0.591 (hour_of_day)	0.753 (RF)	+0.162

The benefit of multi-feature modelling is even more pronounced on *r/pennystocks*, where the strongest individual feature achieves an AUC of just 0.591. Here, multi-feature models improve performance by +0.162 AUC, turning an otherwise weak signal into a viable predictive one.

5.2.4 Cross-Dataset Transfer

Models were evaluated on the other community’s test set without retraining (Table 40). Transfer is strongly asymmetric, and XGBoost transfers best in both directions; the asymmetry is analysed in Section 5.3.4.

Table 40: Cross-dataset transfer AUC-ROC (no retraining).

Direction	LR	RF	XGBoost
WSB-trained to pennystocks test	0.652	0.676	0.684
pennystocks-trained to WSB test	0.753	0.842	0.871

5.2.5 Feature Importance

Table 41: Top-5 permutation importances (10 repeats, scoring=roc_auc) for tree-based models.

Rank	WSB – RF	WSB – XGB	pennystocks – RF	pennystocks – XGB
1	sentiment (+0.146)	sentiment (+0.203)	sentiment (+0.113)	sentiment (+0.165)
2	post_rate_24h (+0.045)	post_rate_24h (+0.067)	time_since_prev (+0.086)	time_since_prev (+0.069)
3	word_count (+0.010)	word_count (+0.027)	word_count (+0.029)	post_rate_24h (+0.015)
4	time_since_prev (+0.005)	time_since_prev (+0.003)	post_rate_24h (+0.023)	word_count (+0.008)
5	accel_x_time (+0.005)	day_of_week (+0.002)	word_count_x_hour (+0.017)	num_tickers (+0.006)

sentiment_score dominates everywhere (+0.113 to +0.203). On WSB, activity features fill ranks 2–3; on pennystocks, time_since_previous rises to second (+0.086), reflecting reliance on temporal gaps when volume is low. Interaction terms remain negligible on WSB ($\leq +0.005$) but word_count_x_hour reaches +0.017 on pennystocks RF.

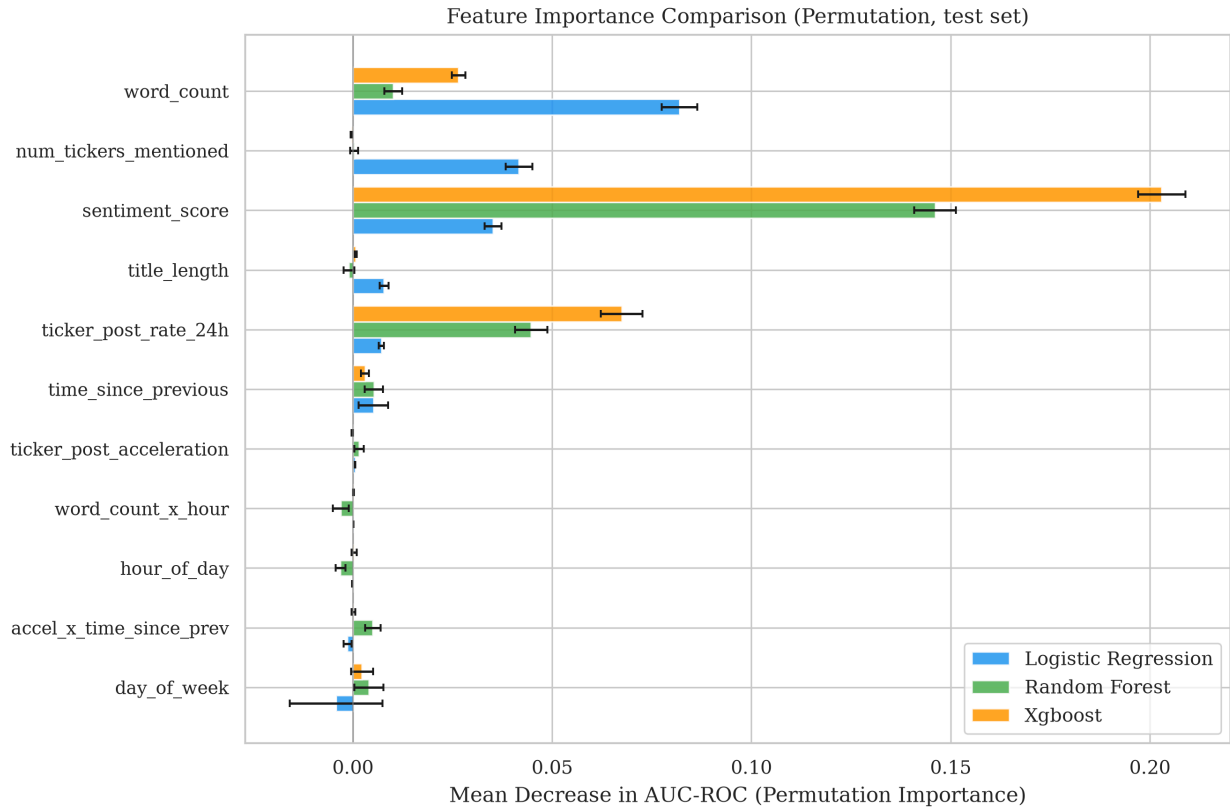


Figure 11: Permutation feature importance (mean decrease in AUC-ROC, 10 repeats) for the three models on the WSB held-out test set. Sentiment score dominates for both tree models, with activity features (ticker_post_rate_24h) next; the cross-community comparison against pennystocks is given in Table 41.

5.2.6 Sentiment Contribution (Phase 1 vs Phase 2)

Does adding sentiment to the surge target actually help, or would volume alone do just as well? To find out, the same XGBoost model was trained on two target definitions: **Phase 1**, volume growth only ($w_2 = 0$), and **Phase 2**, the composite of volume and sentiment ($w_1 = w_2 = 0.5$). Table 42 compares them.

Table 42: Volume-only ($w_2 = 0$) vs composite ($w_1 = w_2 = 0.5$), XGBoost AUC-ROC.

Dataset	Phase 1 (volume only)	Phase 2 (composite)	Δ AUC
WSB	0.710	0.892	+0.182
pennystocks	0.685	0.734	+0.049

Sentiment clearly helps, adding +0.182 AUC on WSB and +0.049 on r/pennystocks. One caveat: changing the weight also shifts the base surge rate (0.53% to 1.44% on WSB), so part of the gain could simply come from a slightly more balanced target rather than from richer signal. To separate the two effects, the full weight sweep varies w_2 from 0 to 1:

Table 43: Weight sensitivity, XGBoost AUC-ROC on WSB ($\tau = 1.5$).

w_2	w_1	AUC-ROC	Surge Rate	Tier
0.00	1.00	0.710	0.53%	Target
0.25	0.75	0.708	1.01%	Target
0.50	0.50	0.892	1.44%	Stretch
0.75	0.25	0.861	4.30%	Stretch
1.00	0.00	0.872	9.62%	Stretch

The sweep confirms the gain is real, not just a balance artefact. Its $w_2 = 0$ row (0.710) is the Phase 1 volume-only result from Table 42; the sharpest move, however, is higher up the sweep. Between $w_2 = 0.25$ (AUC 0.708) and $w_2 = 0.50$ (0.892), AUC leaps +0.184 while the surge rate rises only 0.43pp, far too small a balance change to explain the jump. Sentiment is carrying genuine predictive signal. The curve is also non-monotonic: pure sentiment ($w_2 = 1.0$, 0.872) beats the volume-heavy $w_2 = 0.75$ (0.861), so an equal volume-sentiment balance ($w_2 = 0.50$) works best.

5.3 Critical Analysis

5.3.1 Model Complexity vs Data Density

XGBoost leads on WSB (0.892 vs. RF's 0.880), whereas Random Forest leads on r/pennystocks (0.753 vs. XGBoost's 0.734). Three factors explain this inversion:

First, XGBoost's sequential boosting requires sufficient positive examples to distinguish signal from noise. WSB provides 5,934 training surges versus 667 in pennystocks (Table 29); with fewer positives, later boosting iterations fit noise, a risk bagging mitigates by averaging independent trees.

Second, Random Forest distributes splits more evenly across features. On pennystocks, Gini importances span six features above 0.06 with none exceeding 0.21, offering robustness when individual signals are unreliable.

Third, XGBoost's probability calibration degrades under extreme imbalance. At 31:1 training ratio in pennystocks (105:1 in test, Table 29), its logistic output saturates near zero, yielding zero positive predictions at threshold 0.5. RF's vote-fraction probabilities produce less extreme skew.

On this evidence, below roughly 5,000 positive training instances Random Forest was the more robust choice than gradient boosting.

5.3.2 The Sentiment Signal

As a standalone feature, `sentiment_score` achieves only modest AUC (0.559 on r/pennystocks, 0.587 on WSB; from the single-feature baselines, Section 5.2.3 and Table 44), yet dominates permutation importance (+0.146 to +0.203). This discrepancy arises because permutation importance measures contribution *in context of other features*. Sentiment becomes highly discriminative when conditioned on activity rates: accelerating volume combined with elevated emotional tone is a stronger surge precursor than either signal alone.

This also explains Phase 1 vs Phase 2 divergence. With sentiment excluded from the target ($w_2 = 0$), volume-only surges exhibit higher variance and prove harder to forecast. Incorporating sentiment ($w_2 \geq 0.50$) yields surges with more structured precursors. While part of the +0.182 AUC gain stems from altered task difficulty, its disproportionate magnitude evidences genuine predictive structure.

The performance ceiling is constrained by VADER’s limitations: it fails on domain-specific semantics (“short” as bearish, “moon” as bullish) and sarcasm. FinBERT (Araci 2019) represents a promising improvement avenue.

5.3.3 Error Analysis

At the best WSB operating point (XGBoost, threshold 0.85), the errors split into 511 missed surges (false negatives) and 565 false alarms (false positives) against 157 true positives (Table 37). The two error types have distinct causes.

False negatives (missed surges) concentrate where the backward signal is weakest. A surge that arrives abruptly, from a ticker with little or no prior 24-hour activity, gives the model almost nothing to condition on: `time_since_previous` is undefined or large, `ticker_post_rate_24h` and `ticker_post_acceleration` are near zero, and the sentiment features carry the entire prediction. These cold-start surges, plausibly driven by external news breaking outside the forum, are structurally invisible to a purely backward-looking, forum-only feature set, and are the dominant missed case given recall of just 0.235.

False positives (false alarms) concentrate where precursors resemble a surge but no surge follows: a burst of accelerating, emotionally charged discussion that fizzles out. Because the model ranks such precursor patterns highly, they surface near the top of the ranked list, which is why precision stays near 0.22 even at a strict threshold. A share of false positives also stems from ticker-extraction noise (Assumption 3): common uppercase tokens mis-read as tickers dilute a ticker’s activity counts.

These patterns are systematic rather than random, and they point back to specific design limits: cold-start misses reflect the deliberate exclusion of external-news and network signals (Table 4), while precursor-driven false alarms reflect the intrinsic difficulty of separating a surge from an aborted build-up with backward features alone. Both bound what any single-threshold operating point can achieve here, motivating the rank-based deployment argued in Section 5.3.5.

5.3.4 Cross-Community Transfer and Generalisability

The pennystocks-trained model (21,549 records) transfers upward to WSB at AUC 0.871, whereas the WSB-trained model (388,149 records) achieves only 0.684 downward. This contradicts the assumption that larger datasets yield better transferability.

Distributional mismatch explains the asymmetry, and it shows up in how a single feature behaves across the two communities (Table 44). On WSB, `word_count` alone achieves AUC 0.805, reflecting a community culture where surges are preceded by lengthy “due diligence” posts; on `r/pennystocks` the same feature manages only 0.573. WSB-trained models lean heavily on this community-specific artefact, so they lose ground when transferred. Conversely, pennystocks-trained models cannot rely on any single dominant feature; they learn broader representations combining sentiment and activity that generalise across domains.

Table 44: Two single-feature AUCs contrasted across communities, drawn from the single-feature baselines (Section 5.2.3). `word_count` is strongly community-specific (dominant on WSB, weak on `r/pennystocks`), whereas `sentiment_score` is modest but stable, which is why the sparse-community model transfers better.

Single feature (AUC-ROC)	WSB	pennystocks
<code>word_count</code>	0.805	0.573
<code>sentiment_score</code>	0.587	0.559

This suggests that, for cross-community use, training on the more signal-constrained community forces broader feature reliance and transfers better than training on the larger one; the practical implication for deployment is taken up in Section 5.4.2.

5.3.5 Operational Precision and False Alarm Rate

The best operating point (XGBoost, threshold 0.85 on WSB) is evaluated against the acceptance criteria defined in Section 3.1.3:

Table 45: Best model evaluated against operational acceptance criteria (Section 3.1.3).

Criterion	Requirement	Achieved	Met?
AUC-ROC	≥ 0.80	0.892	Yes
Recall	≥ 0.50	0.235	No
Precision	≥ 0.10	0.217	Yes
Daily alert volume	20–30	~3.7 flags/day*	Below target

* With 722 total flags (157 TP + 565 FP) spread across the 197-day test partition (2021-06-18 to 2021-12-31), the daily average is ~3.7 flags.

The system meets ranking quality and precision requirements but catches only 23.5% of surges rather than the targeted 50%. Lowering the threshold to achieve recall ≥ 0.50 (at default 0.50: recall = 0.819) produces ~65 flags/day (12,843 flags over 197 days), operationally unusable. No single threshold simultaneously satisfies all three criteria at 102:1 class imbalance.

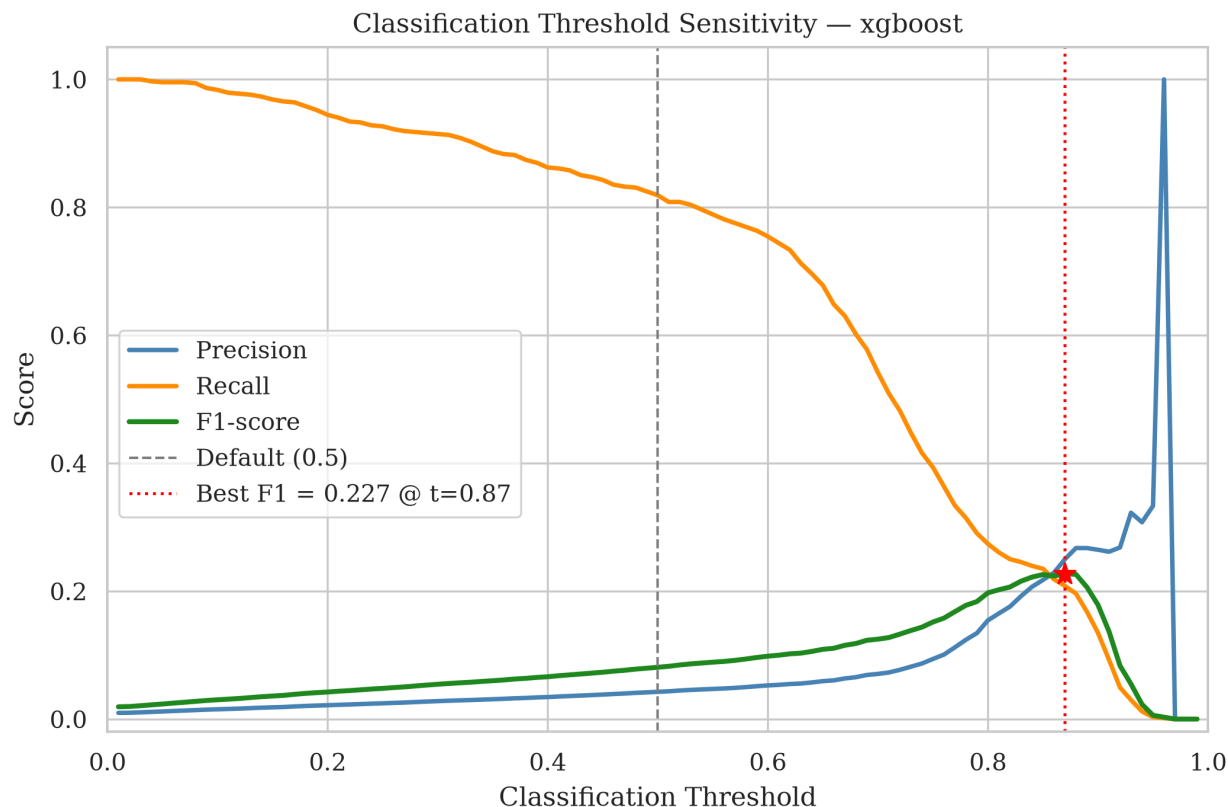


Figure 12: Classification threshold sensitivity for XGBoost on WSB. As the decision threshold varies, precision and recall trade off sharply. No single threshold simultaneously achieves recall ≥ 0.50 and precision ≥ 0.10 , illustrating the fundamental constraint imposed by the 102:1 class imbalance.

Implications by user scenario. *Compliance teams:* insufficient as standalone surveillance, but complementary to rule-based volume alerts. *Quantitative researchers:* well-suited, at ~ 0.8 genuine surges surfaced daily among ~ 3.7 flags (precision 0.217). *Platform moderators:* rank-based deployment (top- k tickers daily) avoids the threshold problem entirely.

Why ranking matters more than binary decisions. The model ranks surges effectively (AUC = 0.892) but struggles to produce calibrated binary predictions at any single threshold, a mathematical consequence of the base rate, not a model failure (He and Garcia 2009). Saito and Rehmsmeier (Saito and Rehmsmeier 2015) demonstrated that under severe imbalance, precision-recall analysis reveals limitations that ROC curves mask. The precision-recall trade-off here confirms this (Figure 12): the model discriminates well across the full score range, yet at the 0.97% base rate a precision of only 0.217 at the best operating point means false alarms dominate at any threshold permissive enough for reasonable recall.

The operationally correct deployment is rank-based: sort tickers by predicted probability, review the top- k daily, and accept that some surges fall outside the review window (Fawcett 2006). Full automation, which as Section 3.1.3 noted would demand far higher precision and formal calibration, is not supported by these results (Section 5.4.2).

5.3.6 Temporal Stability and Distribution Shift

The validation-test gap for Random Forest (val_F1 = 0.911 at tuned threshold vs test F1 = 0.079 at that same threshold, Table 36) suggests temporal non-stationarity, where surge dynamics shifted as the post-GameStop wave subsided. The gap persists at the default threshold (test F1 = 0.145, Table 35). Importantly,

AUC remains high (0.880) on the test set, indicating that ranking ability transfers intact; the F1 collapse reflects threshold miscalibration under distribution shift rather than wholesale model failure. This distinction matters: overfitting would degrade both AUC and F1, whereas a shift in class balance or surge characteristics affects only the calibrated threshold. A deployed system would need periodic retraining or adaptive threshold selection. However, cross-dataset transfer at 0.871 suggests core patterns are stable enough to cross community boundaries; instability is concentrated in threshold calibration rather than underlying ranking.

5.4 Limitations and Proposed Improvements

5.4.1 Limitations

Sample size on pennystocks: 31 test surges yield bootstrap CIs spanning ± 0.08 – 0.09 in AUC with overlap between models, so conclusions from pennystocks alone are tentative.

Single calendar year (2021) including the GameStop episode: models may have learned regime-specific patterns. Running on 2020 or 2022 data would test generality.

Structural correlation between target and top feature: `sentiment_score` dominates importance, but sentiment change is part of the composite target. The feature uses *current* sentiment while the target uses *forward-window* shift, not leakage, but a circularity that likely inflates sentiment’s apparent importance. Phase 1 provides a partial control: with $w_2 = 0$, XGBoost still achieves AUC 0.710 (Table 42).

Smaller concerns: survivorship bias (deleted posts absent); fixed temporal split (~June 2021) makes test difficulty regime-dependent; training variance only partially characterised (0.019 AUC range across 5 seeds).

Class imbalance as deployment constraint: The 102:1 ratio fundamentally constrains what the system can deliver. Even a perfect ranker ($\text{AUC} = 1.0$) faces a precision ceiling when forced into binary decisions at sub-1% prevalence. Surge prediction at this level must operate as either (a) a ranking layer with human top- k review, or (b) a first-stage filter combined with a higher-precision second stage (Saito and Rehmsmeier 2015; He and Garcia 2009). Neither metric alone captures operational value; both must be reported with deployment context. For communities with higher surge prevalence (e.g., $\tau = 1.0$ producing ~5% surge rate), precision constraints relax substantially, suggesting the “right” surge definition should be co-designed with end-users based on alert-handling capacity.

5.4.2 Proposed Improvements

Table 46: Proposed improvements, ranked by priority.

Improvement	Motivation (from results)	Effort	Priority
FinBERT for sentiment	Top feature (+0.203) but VADER misses financial semantics (Section 5.3.2)	Medium	High
Multi-scale windows (6h, 12h, 24h, 72h)	Fixed 24h window may miss faster/slower surges	Medium	High
Probability calibration (Platt/isotonic)	Precision collapse at default threshold is a calibration problem	Low	High
Known-ticker validation list	Ticker heuristics admit false positives diluting activity counts	Low	Medium

Improvement	Motivation (from results)	Effort	Priority
Additional time periods (2020, 2022)	Single-year limitation	Medium	Medium
Additional subreddits (r/stocks, r/investing)	Tests density gradient more granularly	Medium	Low

6 Conclusion (824/1000 words)

6.1 Current Achievements

This project set out to answer a question the existing literature leaves open: can volume and sentiment surges in Reddit financial communities be predicted using only information available at the exact moment of post creation? To avoid the future-engagement leakage common in prior work, such as reliance on post-hoc upvote or comment counts, the pipeline enforces strict temporal ordering across surge definition, feature extraction, and model evaluation.

The resulting framework evaluates raw Reddit data through statistically validated classifiers across two communities, three models, and over thirty experimental runs spanning the planned matrix (Table 21). It answers the question in the affirmative; the subsections below set out the contributions and what they reveal.

6.2 Originality and Contribution

This project makes three contributions.

First, a **leakage-free methodology applied where neglected**. The literature review (Section 2.6, Table 6) shows temporal leakage is the norm in social media prediction studies. This project applies established temporal evaluation principles (Tashman 2000; Bergmeir and Benítez 2012) end-to-end, showing that performance of $AUC = 0.753\text{--}0.892$ is both achievable and trustworthy. The framework is reusable for any timestamped prediction problem.

Second, a **composite surge metric** integrating normalised volume growth with sentiment change, fully parameterised by threshold and weights. The Phase 1 vs Phase 2 experiment (Table 42) confirms it captures a richer phenomenon than volume alone (+0.182 AUC on WSB).

Third, **empirical evidence that data density is the binding constraint**. Same pipeline, same models, different community size: the gap between datasets (0.753 vs 0.892) and asymmetric transfer (sparse to dense at 0.871; dense to sparse at 0.684) demonstrate this clearly.

Direct comparison with published baselines is not possible, as no reviewed study predicts surges on the same datasets with the same temporal protocol. These are incremental contributions, combining established techniques into a coherent framework for a problem prior work has not directly addressed, with each claim grounded in quantified evidence.

6.3 Key Findings

Surges can be predicted from backward-looking signals alone, but how well depends almost entirely on how much data the community generates: both tree models reached the stretch tier on WSB (AUC 0.892

and 0.880), while on sparser r/pennystocks Random Forest managed only 0.753 over just 31 test surges. Beyond that headline, three findings were not obvious going in:

Having more data matters more than having a better model. The gap between datasets (13–14 AUC points) is larger than the gap between any two models on the same dataset. For surge detection on smaller communities, invest in data collection before model tuning.

The advantage of more complex models is not guaranteed. On WSB, gradient boosting earns its complexity. On pennystocks the picture flips: Random Forest beats XGBoost. With only a few hundred positive examples, boosting’s sequential corrections fit noise, while averaging independent trees is more forgiving.

Sentiment does more than add a useful feature; it changes what a surge is. When sentiment is stripped from the target definition, XGBoost’s AUC on WSB drops 18 points. Sentiment is the most important feature by permutation importance, yet weak as a standalone predictor. The signal is interactive: accelerating discussion combined with rising emotional intensity is a meaningful pattern, but either one alone is not.

Transfer between communities is asymmetric. A pennystocks-trained model scores AUC 0.871 on WSB, nearly its native result, but the reverse direction reaches only 0.684. WSB models lean on word count (long “due diligence” posts precede surges there), a cue absent elsewhere; models trained on sparse data spread their reliance across weaker signals and generalise better.

For teams monitoring financial communities, the practical takeaway is to invest in data coverage before model sophistication.

6.4 Limitations and Future Work

All three objectives were met (Section 5.1), but four limitations bound the conclusions.

The pennystocks evaluation rests on thin ground. Thirty-one test surges produce confidence intervals wide enough that model rankings could shift with a different test period.

2021 may not be representative. The GameStop episode makes this one of the most unusual periods of retail speculation in memory. Running on 2020 or 2022 data would matter more for credibility than any model improvement.

VADER does not speak Reddit finance. It misreads domain-specific terms (“short” as negative, “moon” as neutral). FinBERT (Araci 2019) would handle this, though applying it across 1.3M records requires GPU infrastructure.

The models rank surges well but flag them poorly. At extreme imbalance, probability outputs compress so much that useful thresholds sit near 0.16. Post-training calibration (Platt scaling or isotonic regression) would make scores interpretable without changing discriminative power.

Two directions would extend the methodology:

Multi-scale temporal windows. Adding parallel windows at 6h, 12h, and 72h would let the model match its horizon to surge timescale and reveal whether findings are partly an artefact of the 24-hour alignment with daily posting rhythms.

Live prediction testing. Everything here is retrospective. Connecting the pipeline to a live stream, where predictions are recorded before outcomes are known, would produce evidence that retrospective evaluation cannot. In deployment, the system would ingest posts in a rolling stream, recompute features hourly, and surface top- k tickers for human review. Building that system is beyond current scope, but nothing in the architecture prevents it.

7 Appendices

7.1 Project Timeline

#	Key Activities	Deliverables	Dependencies	APRIL		MAY				JUNE				JULY				AUGUST				SEPTEMBER			
				W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
1	Define problem, users, research question, scope and success criteria	Project definition and requirements																							
2	Review trend prediction, engagement prediction and sentiment analysis research	Literature review and research gap	Business Understanding																						
3	Dataset investigation, exploratory analysis and quality assessment	Dataset profile and EDA results	Literature Review																						
4	Design prediction pipeline, feature set, surge definition and evaluation strategy	System architecture and design specification	Data Understanding																						
5	Implement baseline pipeline and Logistic Regression model	Feasibility prototype	System Design																						
6	Preprocessing, feature engineering and advanced models	Complete predictive system	Prototype Development																						
7	Model comparison, cross-validation, error analysis and feature importance analysis	Evaluation results	Full Pipeline Implementation																						
8	Improve models and address identified weaknesses	Refined prototype	Evaluation & Analysis																						
9	Final documentation and demonstration video	Final report and video	Refinement																						

Figure 13: Project Timeline (Gantt Chart).

7.2 Software Dependencies

Table 47 lists the declared dependencies of the surge-pipeline package, taken from `pyproject.toml` and `requirements.txt`. All are specified as minimum-version constraints; the exact resolved versions are captured in each run’s experiment log (Section 4.7). The project targets Python ≥ 3.10 and is built with `setuptools` (≥ 68.0).

Table 47: Declared software dependencies of the surge-pipeline package, from `pyproject.toml` and `requirements.txt`.

Package	Constraint	Group	Role
pandas	≥ 2.0	Runtime	DataFrame ingestion, windowing, and labelling
numpy	≥ 1.24	Runtime	Vectorised counts, z-scores, array operations
scikit-learn	≥ 1.3	Runtime	Logistic Regression, Random Forest, metrics, scaling
xgboost	≥ 1.7	Runtime	Gradient-boosted tree classifier
scipy	≥ 1.10	Runtime	Statistical tests (McNemar’s) and distributions
vaderSentiment	≥ 3.3	Runtime	Rule-based sentiment scoring

Package	Constraint	Group	Role
textblob	≥ 0.17	Runtime	Secondary sentiment signal for EDA agreement check
matplotlib	≥ 3.7	Runtime	Figure generation (ROC, confusion matrices)
seaborn	≥ 0.12	Runtime	Statistical plotting
joblib	≥ 1.2	Runtime	Model persistence
tqdm	≥ 4.60	Runtime	Progress reporting for long-running stages
kaggle	≥ 1.6	EDA	Dataset discovery via the Kaggle API
huggingface_hub	≥ 0.20	EDA	Dataset discovery via the HuggingFace API
pytest	≥ 7.0	Dev	Test suite execution
mypy	≥ 1.10	Dev	Static type checking
ruff	≥ 0.4	Dev	Linting and formatting
pandas-stubs	≥ 2.0	Dev	Type stubs for pandas
nltk	≥ 3.8	Dev	Build-time only: English base of the ticker stopword lexicon

7.3 Repository Structure

Listing 11 gives the full repository code organisation summarised in the Implementation section (Code Organisation). The `surge_pipeline/` package contains one module per pipeline stage (plus a few shared-utility and data-contract modules), enforcing separation of concerns. Each stage module has a corresponding test file. CLI entry points orchestrate multi-stage runs without embedding logic themselves. The `eda/` notebooks sit outside `src/` and share no code with the pipeline package.

Listing 11: Repository code organisation. The `'surge_pipeline/'` package contains one module per pipeline stage (plus a few shared-utility and data-contract modules), enforcing separation of concerns. Each stage module has a corresponding test file. CLI entry points orchestrate multi-stage runs without embedding logic themselves. The `'eda/'` notebooks sit outside `'src/'` and share no code with the pipeline package.

```

eda/
├── 01_discovery.ipynb      # Standalone EDA notebooks
├── 02_highlevel_eval.ipynb # Candidate discovery (Kaggle + HuggingFace APIs)
├── 03_deep_assessment.ipynb # High-level comparative profiling
├── input/                  # Deep viability assessment
├── output/                 # Manually downloaded candidate datasets
├──                         # Screening CSVs + figures
src/
├── surge_pipeline/        # Core library (16 modules, ~4,450 LOC)
│   ├── config.py          # Configuration dataclass + JSON I/O
│   ├── data/              #
│   │   └── ticker_stopwords.txt # stopword lexicon (NLTK base + supplement)
│   ├── loader.py          # CSV ingestion, ticker extraction, explosion
│   ├── windowing.py       # Per-ticker 24h counts (searchsorted)
│   └── sentiment.py       # VADER scoring with title-fallback

```

```

| | — labelling.py          # Temporal split, z-scores, thresholding
| | — normalisation.py     # Z-score parameter persistence
| | — features.py          # 11 backward-only features
| | — training.py          # Expanding-window CV + grid search
| | — training_models.py   # Training result / model-container dataclasses
| | — evaluation.py        # Metrics, bootstrap CI, McNemar's
| | — evaluation_models.py # Evaluation result dataclasses + tier constants
| | — evaluation_figures.py # ROC curves, confusion matrices, plots
| | — experiment_log.py    # Append-only JSONL tracker
| | — timestamps.py       # Portable datetime -> epoch-seconds conversion
| | — cli_logging.py       # Tee-style console + log-file output
| | — pipeline.py          # Orchestrator: chains all stages
| — tests/                # 10 test modules (pytest)
| — run_labeling.py        # CLI: full labelling pipeline (stages 1–4)
| — run_training.py        # CLI: model training + evaluation (stages 5–6)
| — run_cross_validation.py # CLI: cross-dataset transfer evaluation
| — generate_figures.py    # CLI: regenerate figures from saved artefacts
| — generate_prediction_examples.py # CLI: worked prediction examples
| — build_stopwords.py     # Regenerates ticker_stopwords.txt

```

8 References

-
- Araci, Dogu. 2019. *FinBERT: Financial Sentiment Analysis with Pre-Trained Language Models*. arXiv preprint. <https://arxiv.org/abs/1908.10063>.
- Bandari, Roja, Sitaram Asur, and Bernardo A. Huberman. 2012. “The Pulse of News in Social Media: Forecasting Popularity.” *Proceedings of the 6th International AAAI Conference on Weblogs and Social Media (ICWSM ‘12)*, 26–33.
- Bergmeir, Christoph, and José M. Benítez. 2012. “On the Use of Cross-Validation for Time Series Predictor Evaluation.” *Information Sciences* 191 (May): 192–213. <https://doi.org/10.1016/j.ins.2011.12.028>.
- Bollen, Johan, Huina Mao, and Xiaojun Zeng. 2011. “Twitter Mood Predicts the Stock Market.” *Journal of Computational Science* 2 (1): 1–8. <https://doi.org/10.1016/j.jocs.2010.12.007>.
- Cheng, Justin, Lada Adamic, P. Alex Dow, Jon M. Kleinberg, and Jure Leskovec. 2014. “Can Cascades Be Predicted?” *Proceedings of the 23rd International Conference on World Wide Web (WWW ‘14)* (New York, NY, USA), 925–36. <https://doi.org/10.1145/2566486.2567997>.
- Costola, Michele, Matteo Iacopini, and Carlo R. M. A. Santagiustina. 2022. “Self-Induced Consensus of Reddit Users to Characterise the GameStop Short Squeeze.” *Scientific Reports* 12 (1): 13780. <https://doi.org/10.1038/s41598-022-17925-2>.
- Elkan, Charles. 2001. “The Foundations of Cost-Sensitive Learning.” *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI ‘01)* (Seattle, WA), 973–78.
- Fawcett, Tom. 2006. “An Introduction to ROC Analysis.” *Pattern Recognition Letters* 27 (8): 861–74. <https://doi.org/10.1016/j.patrec.2005.10.010>.

- Fernández-Delgado, Manuel, Eva Cernadas, Senén Barro, and Dinani Amorim. 2014. “Do We Need Hundreds of Classifiers to Solve Real World Classification Problems?” *Journal of Machine Learning Research* 15 (1): 3133–81.
- FINRA. 2025. *Social Media-Influenced Investing*. Financial Industry Regulatory Authority. <https://www.finra.org/rules-guidance/key-topics/fintech/report/social-media-influenced-investing>.
- He, Haibo, and Edwardo A. Garcia. 2009. “Learning from Imbalanced Data.” *IEEE Transactions on Knowledge and Data Engineering* 21 (9): 1263–84. <https://doi.org/10.1109/TKDE.2008.239>.
- Hutto, Clayton J., and Eric Gilbert. 2014. “VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text.” *Proceedings of the 8th International AAAI Conference on Weblogs and Social Media (ICWSM '14)* (Ann Arbor, MI), 216–25.
- Kong, Shoubin, Qiaozhu Mei, Ling Feng, Fei Ye, and Zhe Zhao. 2014. “Predicting Bursts and Popularity of Hashtags in Real-Time.” *Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '14)* (New York, NY, USA), 927–30. <https://doi.org/10.1145/2600428.2609476>.
- Lerman, Kristina, and Tad Hogg. 2010. “Using a Model of Social Dynamics to Predict Popularity of News.” *Proceedings of the 19th International Conference on World Wide Web (WWW '10)* (New York, NY, USA), 621–30. <https://doi.org/10.1145/1772690.1772754>.
- Leukipp. 2021. *Reddit Finance Data*. Kaggle. <https://www.kaggle.com/datasets/leukipp/reddit-finance-data>.
- Long, Cathy, Brian Lucey, and Larisa Yarovaya. 2023. “I Just Like the Stock: The Role of Reddit Sentiment in the GameStop Share Rally.” *Financial Review* 58 (1): 19–37. <https://doi.org/10.1111/fire.12328>.
- Mancini, Adriano, Aldo Desiderio, Brendan Marafino, and Alessandro Navigli. 2024. “Detecting Pump and Dump Stock Market Manipulation from Online Forums.” *Digital Finance* 6: 365–93. <https://doi.org/10.1007/s42521-024-00113-6>.
- Saito, Takaya, and Marc Rehmsmeier. 2015. “The Precision-Recall Plot Is More Informative Than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets.” *PLoS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Szabo, Gabor, and Bernardo A. Huberman. 2010. “Predicting the Popularity of Online Content.” *Communications of the ACM* 53 (8): 80–88. <https://doi.org/10.1145/1787234.1787254>.
- Tashman, Leonard J. 2000. “Out-of-Sample Tests of Forecasting Accuracy: An Analysis and Review.” *International Journal of Forecasting* 16 (4): 437–50. [https://doi.org/10.1016/S0169-2070\(00\)00065-0](https://doi.org/10.1016/S0169-2070(00)00065-0).
- Vickers, Andrew J., and Elena B. Elkin. 2006. “Decision Curve Analysis: A Novel Method for Evaluating Prediction Models.” *Medical Decision Making* 26 (6): 565–74. <https://doi.org/10.1177/0272989X06295361>.
- Wang, Fang, and Bernardo A. Huberman. 2013. “Quantifying Long-Term Scientific Impact.” *Science* 342 (6154): 127–32. <https://doi.org/10.1126/science.1237825>.
- Yuan, Chao, and Wentao Li. 2019. “Forecasting the Development Trend of Early-Stage Information Diffusion Based on Empirical Data.” *Physica A* 524 (June): 157–67. <https://doi.org/10.1016/j.physa.2019.04.053>.